AI Director (FramePilot)

Work explained: 8 - 9 October 2026

Everything that changed since the last commit (4a97784): the problems we had, how we fixed them, the real code, the measured results and what the team has to do next.

What	Before	Now
Perception stage (10-min session)	13 - 17 min	3.3 - 5.2 min
AI Brain stage (s03)	41 min	4.2 min
Speaker coverage s01 / s02 / s03	79.8 / 75.8 / 46.3 %	87.0 / 92.1 / 85.9 %
Speaker map right (s03)	8 % (old rule)	100 %
Close-up faces found (s03 camera 2)	22 %	100 %
English words in the s03 transcript	0	about 150
Main video size (10 min)	716 MB	about 300 MB
Speaker on screen with the new split screen (s01 / s02)	87.0 / 92.1 %	91.3 / 95.0 %
Vertical clips: talker left outside the strip (s01 / s03)	16.8 / 11.3 %	0.9 / 0.5 %
Editor Agent test set (real model)	14/15 (3 clips per request)	15/15 (1 clip per request)
Automatic tests	109 passed (8 Oct)	150 passed

Prepared by Ahmed Salama (team lead) for the team: Sara Walid, Nada Ali, Ahmed Sameh, Eslam ElKharbotly
9 October 2026

Contents

0. How to read this document **4**

1. Summary: problems, fixes and gains **4**

1.1 Every problem, how we fixed it, and what we gained . . . 4

1.2 Problems we ran into while working, and what we learned . . . 6

1.3 Main numbers before and after . . . 6

2. Editor Agent (8-9 October) **7**

2.1 The clip length, the reply language, titles and captions . . . 7

2.2 Who to show, and how much . . . 8

2.3 Replies, number of clips, names, follow-ups and a time limit (9 October) . . . 9

2.4 The real test after the fixes, and what it showed (9 October) . . . 10

2.5 Grow the found moment over its topic (9 October) . . . 12

2.6 Why the code reads some things, and not the LLM . . . 13

3. Audio: speakers, captions, laughter, English words, translation **14**

3.1 One voice with two labels (8 October) . . . 14

3.2 Captions: one at a time, right to left (8 October) . . . 14

3.3 Laughter (8 October) . . . 15

3.4 English words in Arabic speech (9 October) . . . 15

3.5 Translation with Gemini (8-9 October) . . . 16

4. Perception: who talks, faces, speed (8-9 October) **18**

4.1 Speaker -> camera map and talking (8 October) . . . 18

4.2 Why perception was slow, and 3 cameras at the same time (9 October) . . . 18

4.3 Head crops: finding faces at panel distance . . . 19

4.4 The speaker map per person . . . 21

4.5 Follow the talker on a two-person camera . . . 22

4.6 Perception time on the full sessions . . . 23

5. Director: the face-size rule and the split screen (9 October) **23**

5.1 The face-size rule in the scorer . . . 23

5.2 Split screen: when to show both close-ups at once . . . 24

5.3 How accurate is it? Tested before building . . . 24

5.4 How it is built . . . 25

5.5 Results . . . 27

6. AI Brain (8-9 October) **28**

6.1 The prompt and laughter from the audio (8 October) . . . 28

6.2 Why the AI Brain took 22-41 minutes (9 October) . . . 28

6.3 How accurate is it? A hand-labelled test . . . 28

6.4 The instruct model + a stricter demo rule . . . 28

7. Render, video quality and highlights **30**

7.1 1080p everywhere (8 October) . . . 30

7.2 Video quality in the config, main video cq 26 (9 October) . . . 30

7.3 Highlight titles with Gemini (9 October) . . . 31

7.4 Vertical clips: the strip moves to whoever talks (9 October) . . . 31

7.5 The 640 px copies removed (9 October) . . . 33

8. Every new or changed setting (configs/default.yaml) **34**

9. Contract changes and what each teammate needs to do **35**
10. Tests **35**
11. How to re-run the sessions (step by step) **36**
12. How to commit this work **36**
13. Not done yet, and next ideas **37**
14. Glossary **37**
Appendix A. Files changed since commit 4a97784 **38**
Appendix B. The full code of the new file english_words.py **40**
Appendix C. The full code of the new file asks.py **43**
Appendix D. The full code of the new file split.py **47**

0. How to read this document

Every part follows the same order, so you always know where you are:

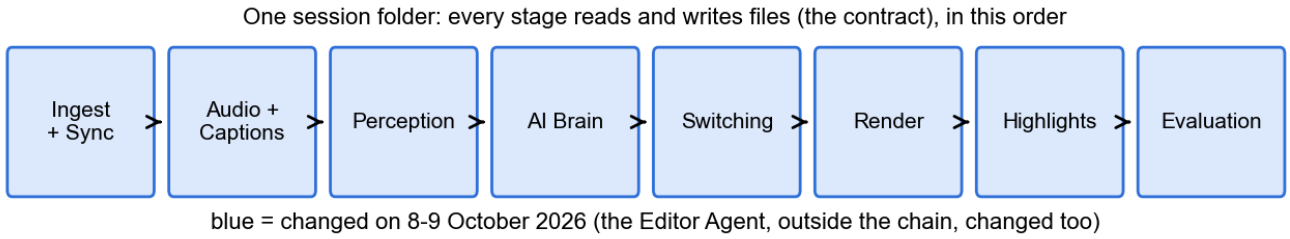
- **Problem**: what was wrong, and how we saw it (a measurement, a screenshot, a test).
- **Fix**: what we changed and in which file.
- **The code**: the real code, copied from the project files when this PDF was made (the numbers on the left are the line numbers in the file), followed by a **line-by-line** table in simple words.
- **What we gained**: the measured result.
- **Done when**: how you can check it yourself.

Where the numbers come from. Everything was measured on our three recordings: **s01** (Egyptian Arabic podcast, 3 speakers, 10 min), **s02** (English interview, 2 speakers, 8 min) and **s03** (Arabic + English panel with a laptop demo, 3 speakers, 10 min). Camera 1 is always the wide camera; cameras 2 and 3 are the close-ups. A **scratch run** means the new code was run on a copy of a session in a temporary folder, so the real session files were not touched; you make the real files when you re-run (part 11).

Hard words are explained in the Glossary (part 14). Nothing in this document is committed to GitHub yet: part 12 shows how to commit it.

1. Summary: problems, fixes and gains

The pipeline runs these stages one after the other. Every stage reads and writes files in one session folder (data/sessions/s01, ...). The names and formats of those files are our **contract**: if one changes, the whole team must know. The blue stages changed on 8-9 October:



1.1 Every problem, how we fixed it, and what we gained

Area	Problem we had	How we fixed it	What we gained
Editor Agent: length	"Make a 1-2 minute clip" gave a 28 s clip (the LLM copied the search result's times; a prompt-only fix did not help)	The CODE reads the length from the request (new agent/asks.py); new *seconds* argument in make_clip	Lengths right; agent test set 14/15 -> 15/15
Editor Agent: language, titles, captions	Replies in the wrong language, Chinese letters in titles, no way to choose captions	asks.py: reply language by words, title checks (rewrite max 2), captions from the request (default: English request -> ar+en, Arabic -> ar)	Wrong-language replies 2/15 -> 0; captions choosable; *clips.json* stores them
Editor Agent: who to show	"Mostly the guest" could not be done	A small LLM question reads who + how much; the code finds the camera boost that gives that share	s01 "mostly the guest, 1.20 minutes" -> 82 s clip, guest on screen 75 %
Speaker labels	Gemini sometimes gives one voice two labels (s01 had 4 labels for 3 people)	Voice fingerprints (WavLM) join the smallest extra label to its voice only if 95 %+ similar	s01: 4 labels -> 3, the right ones
Captions	Up to 5 captions on screen at the same time, some flashing for a split second	One caption at a time: tiny pieces joined, flashes absorbed, each caption ends where the next starts	Max 1 caption at once, none shorter than 0.7 s
Laughter	The detector listened for one laughter type only; the editor never used audio laughter	All laughter types, 3 s windows; heard laughter -> "reaction" mode	s01 laughs found (score 0.16 -> 0.48) -> reaction shots

Area	Problem we had	How we fixed it	What we gained
English words (Arabic sessions)	Gemini Transcribe DROPS English words or writes them in Arabic letters	A general Gemini model listens again with the draft and only adds/fixes the English words	s03: 0 -> about 150 English words (LLM, VLM, Qwen, accuracy, GPT confirmed by Ahmed)
Translation	qwen3:8b made real mistakes ("admins", "art schools", "AI" for API, "Empty")	Gemini 3.1 Flash-Lite with context lines; qwen only when Gemini can't be used	Better on 19 of 37 sample lines, worse on 1; no GPU needed
Speaker -> camera map	The old rule put every s03 speaker on camera 3 (8 % of the speech right)	"Lift" map (8 Oct), then per PERSON (9 Oct)	100 % right on s01, s02, s03
Faces at panel distance	The face model squeezes the whole frame to 128x128: s03 camera 2 found faces in 22 % of the samples	Head crops: cut each head out of the full 1080p frame first	Close-up faces found 100 % in all sessions
Two people on one camera	The camera's face = the biggest face, often the LISTENER	Follow the talker: use the face of the person who is talking	Talker's face used 14 -> 99 % (s01), 34 -> 93 % (s03)
Perception speed	13-17 min per session	3 cameras at the same time, the frame shrunk once	3.3-5.2 min, the same numbers
Scorer	A small face that WAS found scored less than "no face found"	Someone on camera counts at least a third	Coverage 87.0 / 92.1 / 85.9 %; listener on screen s03 12.5 -> 3.3 %
AI Brain speed	13-20 s per moment: the model was secretly "thinking" (550-2,800 hidden words)	qwen3-vl:4b-instruct (the non-thinking version)	1.6 s per moment: s03 41 -> 4.2 min
AI Brain accuracy	False "demo": a laptop lying on the table, or people only TALKING about an app	A stricter demo rule in the prompt + the instruct model	26 test moments: 17 -> 24 right; no false demo in s01/s02
Video size	program.mp4 was 716 MB for 10 minutes	Quality setting in the config, main video cq 26	About -58 % size, VMAF 96 (looks the same)
Resolution	Outputs were 720p	Camera copies and every output in 1080p	Full HD main video, highlights and clips
Highlight titles	qwen3:8b's titles were weak	Gemini 3.1 Flash-Lite writes them (qwen as backup)	1 request per session (not measured on a real session yet)
Agent replies	The reply listed clips that were never made, wrong captions, SPEAKER_01 instead of a name	The CODE writes the list of clips really made; the LLM writes one friendly sentence	Every fact in the reply is true
Agent: number of clips	"Make a reel about ..." made 3 clips	The code reads the number asked (1 if none)	Exactly the number asked
Agent: names	Names are often blank; "Islam" in an Arabic reply became الإسلام	Name, else the role ("the guest"), numbered when two share it, else "speaker 2"	People called the way a viewer would
Agent: follow-ups	"Make it vertical / longer" did not work	Earlier clips in the prompt; the code reads longer / shorter / +20 s and the format	Follow-ups work in Arabic and English
Agent: stuck model	A slow Ollama answer made the chat wait forever	120 s time limit (agent.timeout)	A clear "took too long" reply
Agent: unclear request	"خلي الفيديو أحسن" (make it better) re-edited the whole video	The code refuses a re-edit with no person and no style -> the agent asks a question	Test 6 right again
Agent: wrong moment	One 30 s search piece was often only part of a talk (tests 4 and 14)	The found piece grows over its neighbours about the same topic (max 90 s)	Agent test set 13/15 -> 15/15
Split screen	In fast back-and-forth the edit cut every 2-3 s between the close-ups	Cameras 3 + 2 side by side in fast exchanges and reactions (tested on hand labels first)	Speaker on screen s01 87.0 -> 91.3 %, s02 92.1 -> 95.0 %
Vertical clips, two people on one camera	The 9:16 strip stayed on one person; the other one was never seen talking	The strip cuts to whoever on that camera talks (pieces >= 2 s)	Talker outside the strip s01 16.8 -> 0.9 %, s03 11.3 -> 0.5 %

Area	Problem we had	How we fixed it	What we gained
640 px copies	3 extra videos per session that nothing used any more	Checked every reader, then removed them from ingest, the contract check and the configs	330 MB less, faster ingest

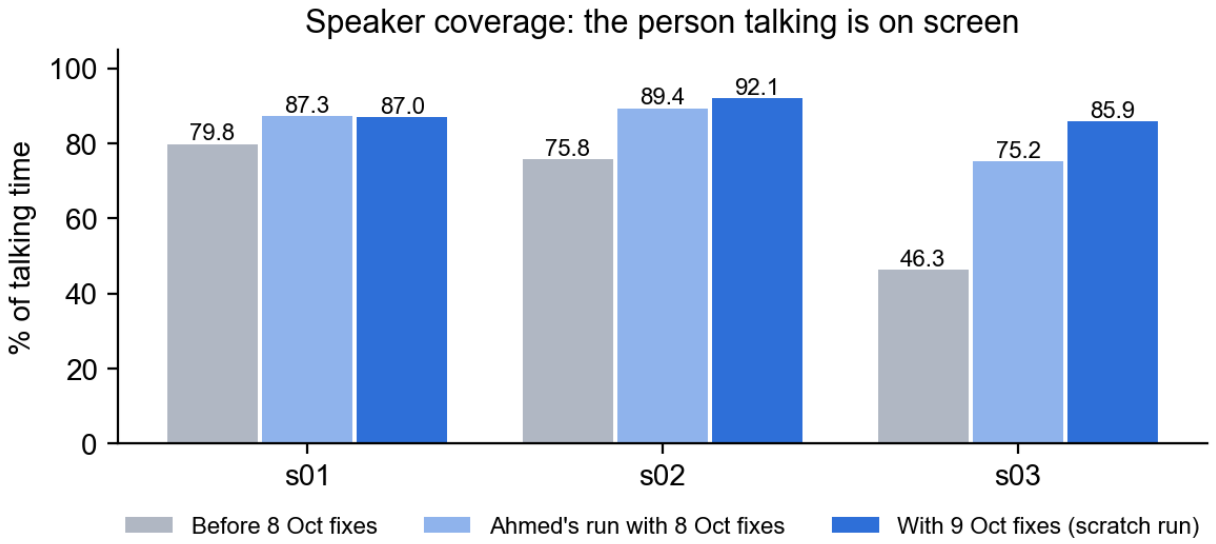
1.2 Problems we ran into while working, and what we learned

What happened	How we found it	What we did / learned
The C: drive was 100 % full (0 bytes free of 465 GB)	A model download stayed at 0 % for 30 min; Ollama's log said "not enough space on the disk"	Freed space (Ahmed). Idea: a pre-flight check that stops the pipeline with a clear message
Gemini free tier: gemini-3.5-flash allows only 20 requests per day per project	429 RESOURCE_EXHAUSTED with "limit 20" in Google's answer	Automatic fallback to gemini-3.1-flash-lite; more keys (= more projects)
One Gemini request hung forever	A run stopped at 9 of 10 windows	180 s time limit per request; a timeout counts as "busy" -> the next key
The first English-words prompt also rewrote the Arabic (12-15 % of the words) and moved words to other lines	Counting kept words; reading s01 3:40	"Minimal correction" prompt + a safety check that ignores answers that look like another line
"Qwen" was heard as "Cohere" at a window edge	Ahmed confirmed the real words	Each window now also gets the 2 lines before/after; names remain the hardest part
A GPU decoding pipe looked faster but was not (at full 1080p)	Timing each step	Dropped it; running the cameras at the same time was the real win
Wrong guess: "the 640 px copy loses faces"	Re-test: it was the face tracker's luck	Corrected in CLAUDE.md; the real fix became head crops
The vision model ignored think=False	Counting answer tokens (550-2,800 for a 40-token answer)	Use the instruct model; measure, don't trust settings
Faces on the wide camera made the wide shot win too often	s03 coverage 78.0 -> 72.9 %	Faces only on the close-ups (config perception.face_roles)
The new LLM "focus reader" added 9 s to every agent request	Timing	It must use the agent's num_ctx; other settings make Ollama reload the model (now 0.5 s)
Test links to the session videos disappeared mid-test	s02/s03 read 0 frames	Hard links instead; perception now stops with a clear error when a video can't be opened
The first real agent test ran while the pipeline used the GPU: 37 s per request	Ollama kept swapping qwen3:8b and qwen3-vl in and out of the GPU	Test LLM features only when the GPU is free (now 13 s per request)
Told "1 clip", the agent asked the search for only 1 moment and took it	The test log: "found 1 moments" every time	The code always returns at least 5 moments to choose from
A prompt-only fix flipped the Arabic "make it better" but not the English one	Trying 5 prompt versions	Small LLMs are unstable: guard important rules in CODE (part 2.6)
The YAML key *on* was read as the value True	A KeyError in the tests	YAML 1.1 treats on/off/yes/no as true/false: the key is now *enabled*
The first "does the face leave the 9:16 strip" count was too high	Looking at the numbers per 3 s: the position jumped between two people	Measure per PERSON, not per camera

1.3 Main numbers before and after

Measure	s01	s02	s03	Notes
Perception time (s): Ahmed's run -> now	1030 -> 313	768 -> 197	940 -> 308	scratch runs on the real videos
AI Brain time (s)	1619	1339	2484 -> 254	new model measured on s03
Speaker coverage (%) before 8 Oct	79.8	75.8	46.3	checked speaker -> camera maps
... Ahmed's run with the 8 Oct fixes	87.3	89.4	75.2	
... with the 9 Oct fixes	87.0	92.1	85.9	scratch run
Listener on screen while someone talks (%)	9.6 -> 10.3	9.0 -> 5.8	12.5 -> 3.3	lower is better
Speaker map right (% of speech)	100	100	87.9 -> 100	

Measure	s01	s02	s03	Notes
Wide shot on screen (%)	3.3 -> 3.0	2.2 -> 3.2	15.2 -> 11.3	
Speaker coverage (%) without -> with split screen	87.0 -> 91.3	92.1 -> 95.0	85.8 (no split)	Ahmed's 9 Oct run, both halves count
Shots in the main video	125 -> 93	90 -> 76	83	fewer ping-pong cuts
9:16: talker outside the strip (%)	16.8 -> 0.9	0 -> 0	11.3 -> 0.5	switching strip



2. Editor Agent (8-9 October)

The Editor Agent is the chat: the user types a wish ("make a 1-2 minute clip about salaries"), a local LLM (qwen3:8b through Ollama) chooses tools (search the transcript, make a clip, ...) and the code renders the clip.

2.1 The clip length, the reply language, titles and captions

Problem

"Make a 1-2 minute clip" gave a 28 s clip: qwen3:8b copied the start and end times of the search result (one search piece is about 30 s). Telling it in the prompt did not help (we tested it).

Some replies came in the wrong language, some titles had Chinese letters or were copied from the transcript, and the user could not choose the captions.

Fix

New file **ai_director/agent/asks.py**: the CODE reads the request instead of trusting the LLM:

- `language()`: Arabic or English by counting words (Franco Arabic counts as Arabic).

- `asked_length()`: "1-2 minutes" -> 60-120 s; دقيقة، دقيقتين، دقيقة ونص، نص دقيقة; "1.20" or "1:20 minutes" = 80 s (like a clock), "1.5" = 90 s, "1 minute 20 seconds" = 80 s.

- `asked_captions()`: none / ar / en / ar+en from the words near caption / subtitle / ترجمة; default: English request -> ar+en, Arabic request -> ar.

- `title_problem()`: Chinese letters, wrong language or copied text -> the LLM writes the title again (max 2 times), else the first words that were said.

- make_clip got a new **seconds** argument: the code sets end = start + length, then moves it to the end of a sentence (+0-5 s).

ai_director/agent/asks.py - asked_length() (lines 60-78)

```
60  def asked_length(message):
61      """The clip length the user typed, as (shortest, longest) seconds, or None.
62      "1-2 minutes" -> (60, 120), "a 30-second clip" -> (30, 30), "120 ,120) <- "اعمل كليب دقيقتين!).
63      Minutes AND seconds are added: "1.20 minutes" (2 digits after the point = seconds, like a clock; "1.5 minutes"
64      stays 90 s), "1:20 minutes", "1 minute 20 seconds", "80 ,80) <- "دقيقة و20 ثانية).""""
65      m = message.lower().translate(str.maketrans("٠١٢٣٤٥٦٧٨٩", "0123456789"))
66      m = CLOCK.sub(lambda x: f"{int(x[1]) * 60 + int(x[2])} sec", m)
67      for pattern, new in WORDS:
68          m = re.sub(pattern, new, m)
69      m = re.sub(r"\b(" + "|".join(sorted(NUMBERS, key=len, reverse=True)) + r")\b",
70                 lambda x: str(NUMBERS[x.group(1)]), m)
71      m = PLUS.sub(lambda x: f"{float(x[1]) * 60 + float(x[2]):g} sec", m)
72      secs = []
73      for n1, u1, n2, u2 in LENGTH.findall(m):
74          per = lambda u: 60 if u.startswith(("min", "دق", "da2")) else 1
75          secs.append(float(n1) * per(u1 or u2))
76          if n2:
77              secs.append(float(n2) * per(u2))
78      return (min(secs), max(secs)) if secs else None
```

Line(s)	What it does, in simple words
60-64	The function and its description: it returns (shortest, longest) seconds, or None if no length was typed.
65	Lower-case the message and turn Arabic digits into normal digits.
66	A clock length ("1:20" / "1.20 minutes") becomes seconds: 1 x 60 + 20 = 80.
67-68	Arabic and Franco length words (دقيقتين, نص دقيقة, ...) become numbers + units.
69-70	Number words ("two", "three", ...) become digits.
71	"1 minute 20 seconds" is added up: 80 sec.
72-77	Every "number (unit)" or "number-number unit" found is turned into seconds (minutes x 60).
78	The shortest and the longest length asked, or None.

2.2 Who to show, and how much

Problem

"Make a clip about salaries, mostly the guest but also the hosts a little" was not possible: the agent could only use the AI edit or one camera.

Fix

read_focus() asks qwen3:8b ONE small JSON question before the agent starts: *which person* and *how much* (not_asked / only / mostly / more). Code turns the level into a share of the clip on that person's camera: only 100 %, mostly 75 %, more 60 % (config agent.focus_share).

boosted_shots() runs the director again with that camera's score multiplied by 1.2, 1.4, ... 3.0 until the share is reached. The LLM chooses the level, the code finds the boost.

It must use the same num_ctx as the agent: otherwise Ollama reloads the model (4 s every time).

ai_director/agent/tools.py - boosted_shots() (lines 254-267)

```
254  def boosted_shots(job, cam, a, b, target):
255      """The AI edit of a..b with camera `cam` boosted just enough to be on screen `target` % of the clip: the smallest
256      boost in configs agent.boosts that reaches it (none if the edit already does), else the one that gets closest.
257      The director (scorer + switcher, 0.2 s for s01) still makes every cut. Returns (shots, {cam: %}, boost)."""
258      best = None
259      for factor in [1.0] + CFG["boosts"]:
260          cuts, _ = decide(job.s, boost={"cam": cam, "factor": factor} if factor > 1 else None)
261          shots = clip_shots(cuts, a, b)
262          share = on_screen(shots)
263          if best is None or share.get(cam, 0) > best[1].get(cam, 0):
264              best = (shots, share, factor)
265          if share.get(cam, 0) >= target:
266              break
267      return best
```

What we gained

Agent test set (eval/agent_tests.json): 14/15 -> 15/15 right, 0 wrong-language replies (was 2/15), 12.6 s per request (+0.9 s for the reader).

Real s01 test "mostly the guest, 1.20 minutes": an 82.4 s clip, boost x1.8, guest 75 % / hosts 17 % / wide 8 %, 16 shots, rendered in 3-4 s.

The focus reader understood 23 of 24 + 9 of 10 test sentences.

Contract change: every item in clips.json now has "captions" ("ar", "en", "ar+en" or "none").

Done when (check it yourself)

Run the agent tests: `.venv\Scripts\python.exe -m pytest -q tests/test_agent.py` (all pass), then ask the agent for a 1-2 minute clip and check the clip length in clips.json.

2.3 Replies, number of clips, names, follow-ups and a time limit (9 October)

Problem

Wrong facts in the reply: the LLM listed clips that were never made (search moments), said "no captions" when the clip had Arabic captions, and wrote SPEAKER_01 instead of a person.

Too many clips: "... اعمل ريل عن" (make A reel) made 3 clips.

Blank names: often nobody has a name (s01: "the host" and "the co-host"), and the Arabic reply turned the name "Islam" into الإسلام (the religion).

Follow-ups ("make it vertical", "make it longer", زود 20 ثانية) did not work, "make a short clip" became vertical, and a stuck Ollama answer made the chat wait forever.

Fix

The code writes the list of the clips really made under the answer (title, id, seconds, format, captions, who is shown); the LLM's text is cut to its first sentence before any list line, file name or link (agent.py: clip_lines, tidy, final_reply).

Number of clips: `asks.asked_count()` reads "3 clips", "two reels", كليبين; no number = 1; a plural without a number ("some clips") = up to 3.

Names: `asks.people_labels()` (below) + `name_people()` replaces every SPEAKER id in the reply.

Follow-ups: the prompt lists the last 5 clips + highlights; a clip that overlaps an earlier one by more than half is allowed without a new search; `length_change()` reads longer / shorter (x1.5, at least 10 s) and "30 seconds longer" / زود 20 ثانية; `asked_format()` reads TikTok / reel / Shorts / عمودي -> 9:16 and YouTube / أفقي -> 16:9 (else the earlier clip's format, else 16:9).

Time limit: one Ollama client with `agent.timeout: 120` seconds -> "the AI model took longer than 120 seconds, try again".

ai_director/agent/asks.py - people_labels() (lines 166-192)

```
166  def people_labels(speakers, lang):
167      """{SPEAKER id: how to call that person in a reply}: the name if there is one (often blank) in the reply's letters,
168      else the role ("the guest" / "الضيف"), numbered when two such people share it ("panelist 1"), else "speaker 2" /
169      "المتحدث 2". An Arabic reply uses the role for a name in English letters ("Islam" became "الإسلام" = the religion)."""
170      def role(p):
171          r = ROLES[lang].get((p["role"] or "").lower())
172          return r if r else None
173
174      def named(p):
175          script = ARABIC if lang == "ar" else LATIN
176          return bool(p["name"]) and (bool(script.search(p["name"])) or not role(p))
177      plain = [role(p) for p in speakers if not named(p)]
178      labels, seen = {}, {}
179      for k, p in enumerate(speakers, 1):
180          if named(p):
181              labels[p["id"]] = p["name"]
182              continue
183          r = role(p)
184          if r and plain.count(r) == 1:
185              labels[p["id"]] = r
186              continue
187          seen[r] = seen.get(r, 0) + 1
188          base = r if r else ("speaker" if lang == "en" else "المتحدث")
189          if lang == "en" and base.startswith("the "):
190              base = base[4:]
191          labels[p["id"]] = f"{base} {seen[r] if r else k}"
192      return labels
```

Line(s)	What it does, in simple words
166-169	The function and its description: {SPEAKER id: how to call that person in the reply}.
170-172	role(p): the person's role in the reply's language ("guest" -> "the guest" / الضيف), or None if there is no role.
174-176	named(p): a name is used only if it is written in the reply's letters (Arabic letters in an Arabic reply), or if there is no role to use instead.
177	The roles of everyone WITHOUT a usable name (to see if two people share a role).
179-182	A usable name -> the name.
183-186	A role that only one person has -> that role ("the host").
187-191	Otherwise a number: "panelist 1", "panelist 2"; no role at all -> "speaker 3" / 3 المتحدث.
192	Give back the whole dictionary.

ai_director/agent/agent.py - final_reply() (lines 128-133)

```
128  def final_reply(text, job, lang, labels):
129      """The answer the user sees: the LLM's friendly words + the clips really made (by the code)."""
130      text = tidy(text, job.clips, labels)
131      if job.clips:
132          return (text or TEXT[lang]["made"]) + "\n" + clip_lines(job.clips, lang, labels)
133      return text or TEXT[lang]["gave_up"]
```

Example reply (real, s03, test 14)

Line 1, written by the LLM: تم إنشاء الكليب "الاشتراك والربح" بنجاح. (the clip was made).

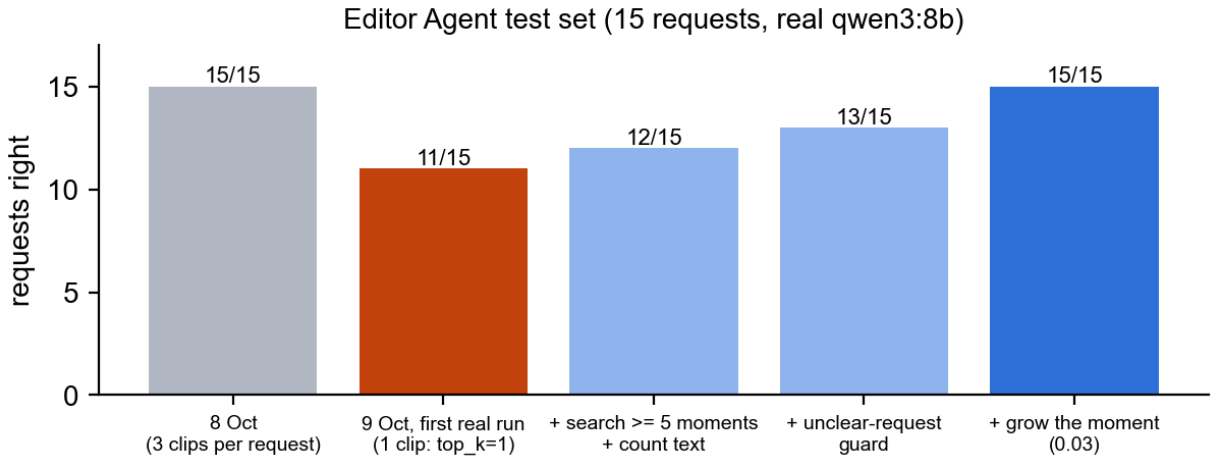
Line 2, written by the code (one line per clip made): the title الاشتراك والربح, the id c1, the length 46 s, the format 16:9 (أفقي = horizontal) and the captions (عربي = Arabic), all in Arabic words.

What we gained

The reply never lists a clip that does not exist, the numbers in it are always right, people are called by name or role, and follow-ups work. 7 new tests in tests/test_agent.py.

2.4 The real test after the fixes, and what it showed (9 October)

The agent test set (eval/agent_tests.json) has 15 requests in Arabic, English and Franco Arabic on s01-s03. A request is right when a clip overlaps the right moment with the right format (and person), or, for an unclear request, when the agent asks a question and makes no clip. Command: `python -m eval.agent_eval --no-render` (decides everything, renders nothing, about 13 s per request).



Problem

The first run gave **11/15**. On 8 October it was 15/15, but then every request made 3 clips, so one of 3 only had to hit the right moment.

Cause 1: told "the user asked for 1 clip", qwen3:8b asked the search for only **1** moment (top_k=1) and took it, so it could not choose the best one.

Cause 2: for "خلي الفيديو أحسن" (make the video better) the same sentence pushed it to act: it re-edited the whole video and even invented "focus on the guest". Changing only the prompt fixed the Arabic request but not the English one (5 prompt versions tried).

Fix

The search tool always returns **at least 5** moments (tools.search_transcript).

The prompt now says "Clips to make for this request: **at most 1 clip (none if the request is unclear)**" (prompts.count_text).

A **code guard** in restyle_video: a whole-video re-edit with no person the user asked for and no real style is refused with "ask the user ONE short question" (real requests like "show the guest more" or "more reactions" still work).

ai_director/agent/tools.py - restyle_video() (lines 404-425)

```
404  def restyle_video(job, a: RestyleArgs):
405      """A new version of the WHOLE video: the director again, with one camera boosted and/or one fixed mode."""
406      if not a.focus_speaker and not a.mode:
407          raise ValueError("give focus_speaker or mode (or both)")
408      person = focus_target(job, a.focus_speaker)[0]                    # nobody asked for -> nobody
409      if not person and a.mode in (None, "conversation") and not CONVERSATION.search(job.message):
410          # "خلي الفيديو أحسن" / "make the video better" became a restyle with an invented person (2026-10-09 eval)
411          raise ValueError("the user named no person and no style, so nothing would change. Do NOT call tools: ask the "
412                           "user ONE short question about what to change (e.g. show someone more, more reactions)")
413      a.focus_speaker = person
414      cam = None
415      if a.focus_speaker:
416          cam = job.speaker(a.focus_speaker)["camera"]
417          if cam is None:
418              raise ValueError(f"{a.focus_speaker} has no close-up camera, so the video can't show them more")
419      if not job.render:
420          return {"file": "program_v2.mp4", "note": "test run: nothing rendered"}
421      cuts = make_cuts(job.s, boost={"cam": cam, "factor": 1.4} if cam else None, force_mode=a.mode, out="cuts_v2.json")
422      render_program(job.s, cuts, out="program_v2_nocaps.mp4")
423      add_captions(job.s, "program_v2_nocaps.mp4", "program_v2.mp4")
424      return {"file": "program_v2.mp4", "shots": len(cuts), "note": "a new version of the whole video "
425                                                                    "(the first version program.mp4 is kept)"}
```

What we gained

11/15 -> 12/15 (search >= 5) -> 13/15 (unclear-request guard). The last two misses were moments next to the right one: part 2.5.

2.5 Grow the found moment over its topic (9 October)

Problem

The search compares the request with ~30 s pieces of the transcript (a new piece every 10 s). One piece is often only PART of a talk. Test 14 (s03, "subscription and profit"): the talk is 271-353 s, the best piece was 359-387 s ("60 percent profit") - on topic, but it starts in the middle and misses the main part. Test 4 (Franco: private universities) was the same.

The idea: the pieces next to the best one score almost as high while the topic is the same, then the score drops when the topic changes. Real scores for test 14:

Piece (s)	330-360	341-369	359-387 (best)	373-402	384-412
Score (similarity)	0.517	0.519	**0.548**	0.503	0.372
Within 0.03 of the best?	no	yes -> joins	-	no	no (new topic)

Fix

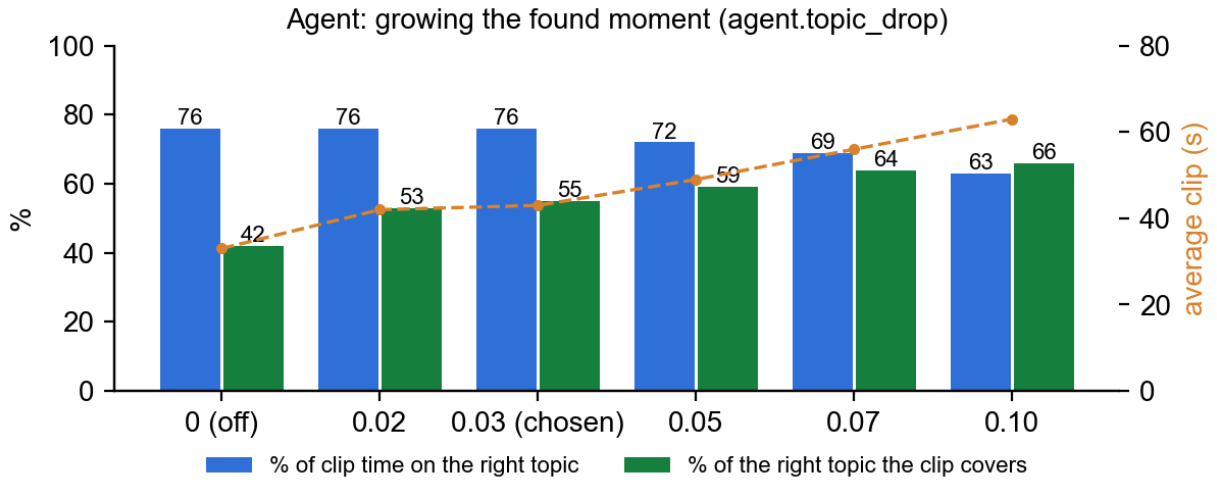
search.topic(): start from the best piece and keep adding the better-scoring neighbour while its score is at most **agent.topic_drop** (0.03) below the best piece's, up to **agent.topic_max** (90 s), never into another found moment. The moment's text stays the best piece's (what the LLM reads).

ai_director/agent/search.py - topic() (lines 74-88)

```
74  def topic(pieces, scores, i, taken=(), drop=None, longest=None):
75      """Piece i grown over its neighbours about the SAME topic: the better neighbour first, while its score is at most
76      agent.topic_drop below piece i's, up to agent.topic_max seconds, never into a moment already found (taken).
77      One 30 s piece is often only part of a talk (2026-10-09: "subscription and profit" got the last 30 s of a 90 s
78      talk). -> (first piece, last piece)."""
79      drop, longest = drop or CFG["topic_drop"], longest or CFG["topic_max"]
80      a = b = i
81      while True:
82          near = [j for j in (a - 1, b + 1) if 0 <= j < len(pieces) and scores[j] >= scores[i] - drop
83                  and pieces[max(b, j)]["end"] - pieces[min(a, j)]["start"] <= longest
84                  and not any(pieces[j]["start"] < m["end"] and m["start"] < pieces[j]["end"] for m in taken)]
85          if not near:
86              return a, b
87          j = max(near, key=lambda j: scores[j])
88          a, b = min(a, j), max(b, j)
```

Line(s)	What it does, in simple words
74-78	The function and its description: piece i grown over its neighbours -> (first piece, last piece).
79	The two settings from the config (0.03 and 90 s), unless a test gives others.
80	Start with the best piece only (a = first piece, b = last piece).
81-84	The neighbours that may join: the piece just before a and just after b, if their score is at most 0.03 below the best, the moment stays at most 90 s long, and they don't overlap a moment already found.
85-86	No neighbour left -> the moment is pieces a..b.
87-88	The better of the two neighbours joins; then look again.

Choosing the number. A longer clip passes the test more easily (it only has to overlap), so we also measured how much of each clip is ON the right topic and how much of the topic it covers:



Real test set (qwen3:8b)	Before	After (0.03)
Requests right	13/15	**15/15**
Average clip length (no length asked)	34 s	40 s
Share of the clip on the right topic	76 %	71 %
Share of the right topic in the clip	42 %	47 %

What we gained

15/15. Tests 4 and 14 now land on the right topic, but only partly (about a quarter of each clip is inside the expected part). Honest note: 0.03 was chosen on these same 15 tests, so 15/15 is a bit optimistic.

When the user asks for a length, the clip still has exactly that length (it starts at the topic's start).

2.6 Why the code reads some things, and not the LLM

Ahmed asked: why do we read the length, format and so on with code instead of letting the LLM do it? Because qwen3:8b is a **small** model on our own GPU: good at **meaning**, unreliable with **exact details** (numbers, counting, copying). Our own tests showed it:

What qwen3:8b did alone	Real example
Wrong length	"1-2 minutes" -> 28 s (it copied the search result); a prompt fix did not help. Reading lengths: the LLM 16/20 test sentences, the code 19-20/20
Wrong count	"اعمل ريل" (one reel) -> 3 clips
Wrong format	"Make a short clip" -> vertical
Wrong facts	Listed clips never made, "no captions" when there were captions, SPEAKER_01, "Islam" -> الإسلام
Unstable	Removing one paragraph of the prompt changed the answer for the Arabic request but not the English one

The rule we follow: the LLM suggests, the code checks

Code: facts written plainly in the message (length, number of clips, format, caption languages) and checks of the LLM's work (no invented times, no extra clips, the real list in the reply). Code is instant, always gives the same answer, needs no GPU and every rule has a test.

LLM: everything that needs understanding: turning the topic into a search, choosing the best moment, who to show (also an LLM), titles, noticing an unclear request, the friendly reply.

It is a mix, not code only: when the message has a length word the code can't read, the LLM's number is used. A much bigger model would make fewer mistakes, but it costs money and needs the internet.

3. Audio: speakers, captions, laughter, English words, translation

3.1 One voice with two labels (8 October)

Problem

Gemini Transcribe sometimes gives the same person two labels (s01: 4 labels for 3 people).

Fix

merge_extra_voices() in audio/voices.py: while there are more labels than people typed at upload, the smallest label joins its closest voice ONLY if the voice fingerprints (WavLM) are at least 95 % similar (config audio.merge_voice: 0.95). Measured: the same person 0.98; two different men 0.89-0.93; so a lower number would join different people. For an existing session: python -m ai_director.audio.voices data/sessions/s01

What we gained

s01: SPEAKER_03 joined SPEAKER_01 (right); s02 and s03 unchanged (right).

3.2 Captions: one at a time, right to left (8 October)

Problem

Up to 5 captions were on the screen at the same time, and some flashed for a split second.

Fix

subtitles.caption_items(): pieces -> **join_tiny()** (a piece shorter than 1 s, within 0.5 s of its neighbour, joins it; max 2 people -> "- " dialogue lines, max 42 characters per line) -> **absorb_flashes()** (a caption the next one would cut off in under 0.7 s joins the next) -> every caption ends where the next one starts.

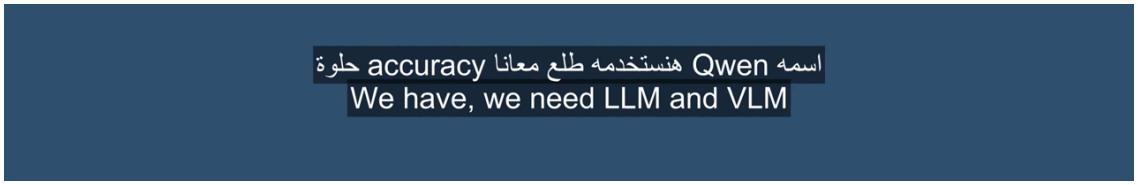
Arabic lines: Encoding=-1 in the style + an invisible right-to-left mark (RLM) in front of every Arabic line, so a line that starts with an English word is still right-to-left.

ai_director/audio/subtitles.py - absorb_flashes() (lines 85-102)

```
 85  def absorb_flashes(groups, gap=0.5, min_show=0.7):
 86      """A short caption that the next one would push off the screen in under `min_show` s ("ليه؟" right before a long
 87      sentence) joins that next caption (a long line simply wraps), so nothing flashes by too fast to read."""
 88      out = []
 89      for k, g in enumerate(groups):
 90          nxt = groups[k + 1] if k + 1 < len(groups) else None
 91          if nxt and min(max(g["end"], g["start"] + 1.0), nxt["start"]) - g["start"] < min_show \
 92                  and nxt["start"] - g["end"] <= gap and len({*g["who"], *nxt["who"]}) <= 2:
 93              for who in reversed(g["who"]):
 94                  if who not in nxt["text"]:
 95                      nxt["who"].insert(0, who)
 96                      nxt["text"][who] = {"ar": [], "en": []}
 97                  for l in ("ar", "en"):
 98                      nxt["text"][who][l][:0] = g["text"][who][l]
 99              nxt["start"] = g["start"]
100              continue
101          out.append(g)
102      return out
```

What we gained

Max 1 caption at a time (was up to 5), max 4 lines (2 people x 2 languages), none shown shorter than 0.7 s. Mixed Arabic + English lines (part 3.4) display in the right order:



3.3 Laughter (8 October)

Problem

The laughter detector (PANNs) listened for one laughter class only, and the editor never used the audio laughter (only the vision model's guess).

Fix

events.laughter(): all laughter classes (Laughter, Giggle, Snicker, Belly laugh, Chuckle, Baby laughter), a 3 s window centred on every second.

brain/modes.py: audio laughter of at least 0.25 (brain.laugh) in the 5 s -> "reaction" at once; the vision model's "laughter" only counts if the audio hears at least 0.1 (brain.laugh_agree) around it.

What we gained

s01: the highest laughter score 0.16 -> 0.48; the laughs at 294-296 s and 359-360 s become reaction shots. s02/s03: at most 0.04 (the model hears no laughter there; Ahmed will send timestamps to check).

3.4 English words in Arabic speech (9 October)

Problem

Gemini 3.5 Transcribe (language ar-EG) **drops** many English words and writes others in Arabic letters. s03, 3:00-3:20 (old -> what was really said):

- اسمه هنستخدمه -> هنستخدمه Qwen اسمه (the model's name was missing)

- هنضطر نرجع ل -> GPT هنضطر نرجع لـ

- طلع معانا ايكورسي حلوه -> حلوة accuracy طلع معانا

Asking Gemini for Arabic + English (or auto) made it WORSE: 101 / 91 words instead of 103.

Fix

New file **ai_director/audio/english_words.py**, step 3 of 7 in the audio stage, Arabic sessions only:

1. The transcript rows are cut into ~1-minute **windows** (a row is never cut).

2. For every window, gemini-3.5-flash gets the window's audio + the numbered draft lines (+ 2 lines before and after as context) and returns every line corrected.

3. The prompt asks for a **minimal correction**: put back the English words, rewrite English written in Arabic letters, keep every other word.

4. A **safety check** ignores an answer that keeps less than half of the line's real Arabic words (it slid to another line).

5. A changed line gets its words spread over its time and an empty translation (it is translated again). Times and speakers never change.

6. Answers are saved in tmp/ (a re-run is free). No quota for 3.5-flash -> gemini-3.1-flash-lite takes over.

ai_director/audio/english_words.py - windows() (lines 50-58)

```
50  def windows(rows, seconds=60.0):
51      """Rows -> groups of neighbouring rows that last at most `seconds` together (a row is never cut)."""
52      out, cur = [], []
53      for r in rows:
54          if cur and r["end"] - cur[0]["start"] > seconds:
55              out.append(cur)
56              cur = []
57          cur.append(r)
58      return out + [cur] if cur else out
```

Line(s)	What it does, in simple words
52	*out* = the finished windows, *cur* = the window being filled.
53-56	Go through the rows in time order. If adding this row would make the window longer than 60 s, the window is finished and a new one starts.

Line(s)	What it does, in simple words
57	The row goes into the current window (so a row is never split in two).
58	Return all windows, including the last one.

ai_director/audio/english_words.py - plausible() (lines 97-101)

```
 97  def plausible(draft, new):
 98      """False if the new text looks like ANOTHER line's: the draft has 3+ real Arabic words (3+ letters, so not في, دة, لـو) and the new text keeps less than half of them."""
 99      old = {w for w in plain(draft) if len(w) >= 3}
100      return len(old) < 3 or len(old & plain(new)) >= len(old) / 2
```

ai_director/audio/english_words.py - spread() (lines 104-111)

```
104  def spread(row, text):
105      """The row with a new text; its words share the row's time evenly (one row = one caption, so only the row's start
106      and end matter). The old translation is removed: the translation step makes it again from the new text."""
107      ws = text.split()
108      step = (row["end"] - row["start"]) / len(ws)
109      words = [{"w": w, "start": round(row["start"] + i * step, 2), "end": round(row["start"] + (i + 1) * step, 2)}
110               for i, w in enumerate(ws)]
111      return {**row, "text": text, "translation": "", "words": words}
```

Line(s)	What it does, in simple words
107	The new text, cut into words.
108	Every word gets the same share of the row's time.
109-110	The word list with start and end times (captions need words with times).
111	The row with the new text and words; translation "" so the translation step makes it again.

Test (s03, same 174 lines)	English words put back	Arabic words kept	3:00-3:20 (LLM, VLM, Qwen, accuracy, GPT)
gemini-3.5-flash (chosen)	152	93.1 %	LLM, VLM, accuracy, GPT (Qwen heard as "Cohere")
gemini-3.1-flash-lite (the fallback)	98	95.4 %	accuracy, GPT
First prompt ("write what is said")	about 150	85.5 % (rewrote too much)	words moved between lines

What we gained

s03: 0 -> about 150 English words in the transcript; s01: 0 -> about 50 (AI, track, base knowledge, junior), with the Arabic kept. Time: 1.5-2 min per session (10-11 requests).

The English captions get better too: the English words no longer have to be guessed back from Arabic letters.

Done when (check it yourself)

python -m ai_director.audio.english_words data/sessions/s03 prints "... lines rewritten, ... English words in the transcript now", and transcript.json at 188 s contains Qwen / accuracy.

3.5 Translation with Gemini (8-9 October)

Problem

8 Oct: rows were translated alone, so a one-word reply lost its meaning (فاضيين -> "Empty"), and English words in Arabic letters came back wrong ("Yousers").

9 Oct: even with context, qwen3:8b made real mistakes:

Arabic	qwen3:8b	gemini-3.1-flash-lite
فاضيين	Empty	You're just idle
انت شايف ان انت واقعي	Are you seeing that you're real	Do you think you're being realistic
البني ادمين دول	these admins	these humans
المدارس الفنيه	the art schools	technical schools
مش هنبدا اي بي اي	not starting any AI	won't start an API

Arabic	qwen3:8b	gemini-3.1-flash-lite
المكسب بعدين مش هيبقى	As they say	The profit later won't be...

Fix

8 Oct: batch_prompt() sends 10 rows as ONE conversation with 2 rows before and after as context, and tells the model that Egyptians write English words in Arabic letters (موديل = model).

9 Oct: **Translator** asks Gemini (audio.translate_model: gemini-3.1-flash-lite) first; when Gemini can't be used (no key, daily limit, no internet) it switches to qwen3:8b for the rest of the session.

ai_director/audio/translate.py - Translator() (lines 91-111)

```
 91  class Translator:
 92      """Asks Gemini (audio.translate_model) while it works, then the local model (ollama.text_model) for the rest.
 93      2026-10-09, s01 + s03 with the same prompt: gemini-3.1-flash-lite better on 19 of 37 sample lines, qwen3:8b on 1
 94      (qwen: "البني ادمين" -> "admins", "المدارس الفنيه" -> "art schools", "اي بي اي" -> "AI", "فاضيين" -> "Empty");
 95      no GPU needed, 1-2 min per session, ~20-26 requests."""
 96      def __init__(self):
 97          self.gemini, self.used = bool(GEMINI_MODEL), set()
 98
 99      def __call__(self, prompt, schema):
100          if self.gemini:
101              try:
102                  answer = ask_gemini(prompt, schema)
103                  self.used.add(GEMINI_MODEL)
104                  return answer
105              except ValidationError:
106                  raise
107              except Exception as e:                                  # no key / limit / no internet: local from now on
108                  print(f"  Gemini translation stopped ({type(e).__name__}: {str(e)[:120]}) -> {TEXT_MODEL} for the rest")
109                  self.gemini = False
110          self.used.add(TEXT_MODEL)
111          return chat_json(TEXT_MODEL, prompt, schema, think=False)
```

Line(s)	What it does, in simple words
96-97	Start with Gemini if a Gemini model is set in the config; remember which models were used.
99-104	Ask Gemini; on success remember that Gemini was used and return the answer.
105-106	A badly formed answer is passed on (the caller asks that line again alone).
107-109	Any other error (no key, limit, no internet): print why and use the local model from now on.
110-111	The local model qwen3:8b answers (the GPU is freed at the end only if it was used).

What we gained

Gemini was better on 19 of 37 sample lines, qwen on 1, the rest the same (Ahmed's reading). 0 missing lines; s01 in 132 s (26 requests), s03 in 63 s (19 requests); no GPU needed.

Gemini free-tier limits (measured on our keys)

gemini-3.5-flash: **20 requests per day per project** (Google's answer: GenerateRequestsPerDay..., limit 20). Each of our keys is its own project.

gemini-3.1-flash-lite: not reached after about 35 requests per key in one day.

Google: limits are per project, not per key; the exact numbers are on aistudio.google.com/rate-limit.

Per session: 1 Transcribe request + about 10 (English words) + 20-26 (translation) + 1 (titles) + 1 (speaker roles).

4. Perception: who talks, faces, speed (8-9 October)

4.1 Speaker -> camera map and talking (8 October)

Problem

The old rule ("the camera whose mouth moves most") gave every s03 speaker to camera 3, the camera with the clearest face: only 8 % of the speech was mapped right.

Fix

Lift: for each speaker, the camera whose mouth moves MORE THAN USUAL while that speaker talks (mouth movement while they talk / while the others talk).

speaker_prob = max(mouth, transcript): the camera of the person the transcript says is talking counts as talking even when the face isn't found (a max was better than a 50/50 or 70/30 average).

YOLO11s-pose: gesture = wrist movement against the person's own body (walking is not a gesture).

What we gained

Map right: 100 % on s01-s03. With the checked maps the coverage went 79.8 -> 86.2 % (s01), 75.8 -> 88.5 % (s02), 46.3 -> 85.4 % (s03) in the 8 Oct test; Ahmed's real run then gave 87.3 / 89.4 / 75.2 %.

4.2 Why perception was slow, and 3 cameras at the same time (9 October)

Problem

After the move to 1080p, perception took 13-17 minutes per session. The cameras were done one after the other, while the PC has 12 CPU threads and the GPU was only ~9 % busy.

Step (camera 2, every 0.1 s of video)	Time
Reading the frame (OpenCV)	3.4 ms
Shrinking the 1080p frame twice (720p for faces, 640 px for pose)	9.0 ms
Pose (YOLO)	25.5 ms
Face (MediaPipe)	14.7 ms
Picture quality	4.1 ms

Fix

The 3 cameras now run **at the same time**, each in its own process (config perception.workers: 3).

The frame is decoded once and shrunk once (640 px); faces come from head crops (4.3).

Tested and dropped: decoding on the GPU through an FFmpeg pipe. At full 1080p it was slower (104.6 s vs 97.7 s): converting and piping a 6 MB frame costs 8.5 ms, OpenCV's reading 3.4 ms.

ai_director/perception/run.py - main() (first part)

```
76  def main(s, seconds=None, workers=None):
77      meta = read_json(s / "session.json")
78      roles = {c["id"]: c["role"] for c in meta["cameras"]}
79      jobs = [(s / f"video/cam{c}.mp4", c, meta["fps"], seconds, roles[c] in CFG["face_roles"]) for c in roles]
80      workers = min(workers or CFG["workers"], len(jobs))
81      t0 = time.time()
82      print(f"  {len(jobs)} cameras, {workers} at the same time ...", flush=True)
83      if workers > 1:                                  # each camera in its own process (own CPU core + GPU share)
84          with ProcessPoolExecutor(workers) as ex:
85              done = [f.result() for f in [ex.submit(process_camera, *j) for j in jobs]]
86      else:
87          done = [process_camera(*j) for j in jobs]
88      print(f"  all cameras done in {time.time() - t0:.0f} s")
89      df = pd.DataFrame([r for rows, _ in done for r in rows])
90      persons = pd.DataFrame([p for _, ps in done for p in ps])
91      who = person_map(s, persons)                     # speaker -> the person whose mouth moves when they talk
92      follow_talker(s, df, persons, who)               # two people on a camera: the face of the one who talks
93      add_speaker_prob(s, df)                          # the mouth ...
94      spk_map = speaker_map(s, df, who)                # (speakers person_map doesn't know: camera-level lift)
95      add_transcript_talk(s, df, spk_map)              # ... or the transcript says this camera's person talks
96      for k, v in who.items():
97          print(f"  {k}: camera {v['cam']}, the person at x = {v['x']} (lift {v['lift']})")
```

Line(s)	What it does, in simple words
77-79	Read the session; one job per camera; faces only for cameras whose role is in perception.face_roles.
80	How many cameras at the same time (config perception.workers, max the number of cameras).
83-85	More than 1: every camera in its own process; wait for all of them.
86-87	1: one after another (the old way).
89-90	Join all cameras' rows, and all the per-person rows.
91-92	Which person is each speaker (4.4); on a 2-person camera use the face of the one who talks (4.5).
93-95	speaker_prob from the mouth, the speaker -> camera map, then the transcript (as on 8 Oct).
96-97	Print where each speaker sits (camera + x position) so you can check it.

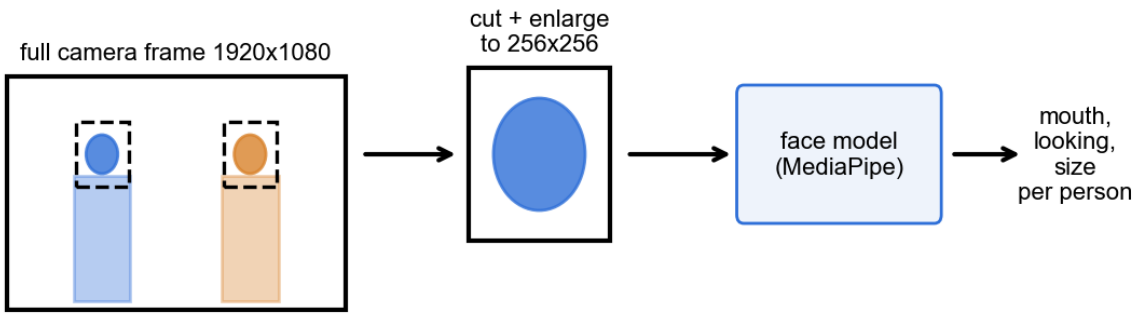
What we gained

60 s of s01: one after another 97.7 s, all 3 at once 37.0 s (2.6x faster) with **100 % identical numbers**. Full sessions in part 4.6.

4.3 Head crops: finding faces at panel distance

Problem

The face model (MediaPipe Face Landmarker) squeezes the WHOLE frame to 128x128 pixels before looking for faces (we read this from the model file: detector 128x128, landmarks 256x256). A head of ~180 px at panel distance became ~12 px. s03 camera 2: faces found in only 22 % of the samples; tracking made the result change from run to run.



YOLO-pose finds nose, eyes, ears -> a square around each head

Fix

YOLO-pose already finds every person's nose, eyes and ears. **head_box()** makes a square around each head, **cut_square()** cuts it out of the FULL 1080p frame, it is enlarged to 256x256 and only then the face model looks at it (one face per crop, no tracking).

ai_director/perception/faces.py - head_box() (lines 31-42)

```
31  def head_box(pts, conf, box, scale=1.0):
32      """Where one person's head is: (centre x, centre y, side of the square) in full-frame pixels. pts/conf/box come
33      from YOLO-pose on a smaller frame (scale = full width / that width). From the face points (nose, eyes, ears) when
34      2+ are seen, else the top of the person's box."""
35      seen = [pts[j] for j in range(5) if conf[j] > 0.3]
36      if len(seen) >= 2:
37          xs, ys = [p[0] for p in seen], [p[1] for p in seen]
38          cx, cy = float(np.mean(xs)), float(np.mean(ys))
39          side = max(max(xs) - min(xs), max(ys) - min(ys), 0.25 * (box[2] - box[0])) * HEAD
40      else:
41          cx, cy, side = (box[0] + box[2]) / 2, box[1] + 0.12 * (box[3] - box[1]), 0.5 * (box[2] - box[0])
42      return cx * scale, cy * scale, max(side * scale, 64)
```

Line(s)	What it does, in simple words
35	The face points (nose, 2 eyes, 2 ears) that YOLO is sure about (more than 30 %).
36-39	With 2+ points: the centre of the points; the square = 2.4 x their spread (at least a quarter of the body's width).
40-41	Too few points: the top of the person's box.
42	Back to full-frame pixels (YOLO ran on a 640 px copy); never smaller than 64 px.

ai_director/perception/faces.py - cut_square() (lines 45-52)

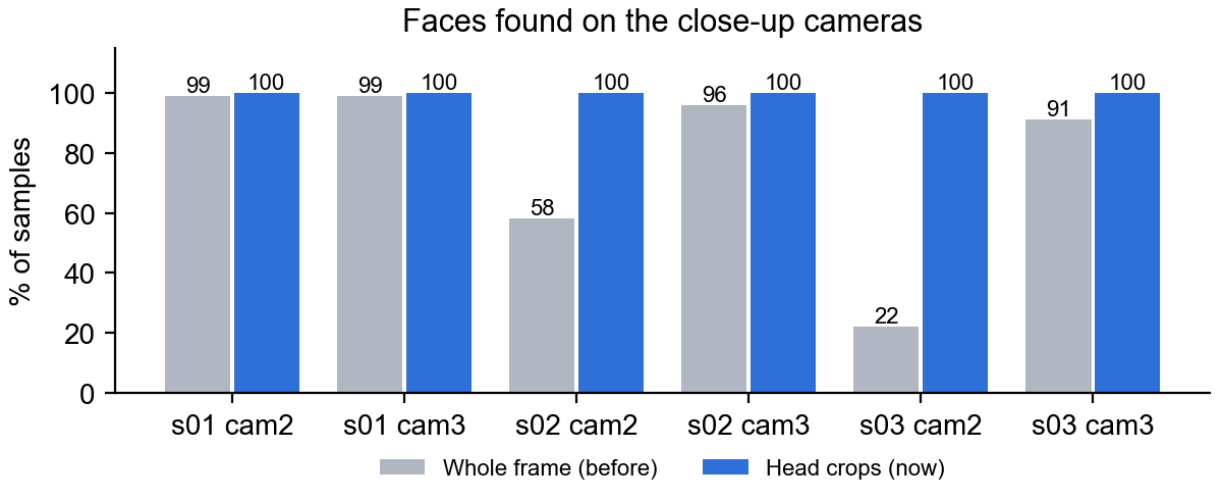
```
45  def cut_square(frame, x0, y0, side):
46      """The side x side square at (x0, y0); where it leaves the frame it is filled with black (stays square, so the
47      face is not stretched). None if it is completely outside."""
48      h, w = frame.shape[:2]
49      a, b, c, d = max(x0, 0), max(y0, 0), min(x0 + side, w), min(y0 + side, h)
50      if c <= a or d <= b:
51          return None
52      return cv2.copyMakeBorder(frame[b:d, a:c], b - y0, y0 + side - d, a - x0, x0 + side - c, cv2.BORDER_CONSTANT)
```

Line(s)	What it does, in simple words
48-49	Where the square and the frame overlap.
50-51	Completely outside the frame: nothing to cut.
52	Cut the overlap and fill the rest with black, so the face is never stretched.

ai_director/perception/faces.py - head_face() (lines 55-70)

```
55  def head_face(lm, frame, cx, cy, side):
56      """One head (from head_box): cut it out of the full frame, enlarge it to 256x256, find the face in it ->
57      face_area, face_x (0..1 of the FULL frame, like before), frontalness, mouth_open; None if no face is found."""
58      side = int(round(side))
59      x0, y0 = int(round(cx - side / 2)), int(round(cy - side / 2))
60      crop = cut_square(frame, x0, y0, side)
61      if crop is None:
62          return None
63      crop = cv2.resize(crop, (CROP, CROP), interpolation=cv2.INTER_LINEAR)
64      res = lm.detect(mp.Image(image_format=mp.ImageFormat.SRGB, data=cv2.cvtColor(crop, cv2.COLOR_BGR2RGB)))
65      if not res.face_landmarks:
66          return None
67      f = face_info(res.face_landmarks[0])
68      h, w = frame.shape[:2]
69      return {"face_area": f["face_area"] * side * side / (w * h), "face_x": (x0 + f["face_x"] * side) / w,
70              "frontalness": f["frontalness"], "mouth_open": f["mouth_open"]}
```

Line(s)	What it does, in simple words
58-62	Cut the head square out of the full frame.
63-64	Enlarge it to 256x256 (the landmark model's own size) and let the face model look at it.
65-66	No face found: None.
67-70	The face numbers, converted back to the FULL frame: face_area as a share of the frame, face_x 0..1, how frontal, how open the mouth is.



What we gained

Close-up faces found: s01 99/99 -> 100/100 %, s02 58/96 -> 100/100 %, s03 22/91 -> 100/100 %.

Cost: about 8-9 ms per head. Faces on the WIDE camera are switched off (config perception.face_roles: [closeup]): they made the wide shot win too often (s03 coverage 78.0 -> 72.9 %, wide 12 -> 21 %).

4.4 The speaker map per person

Problem

On s03 camera 2 the presenter and the woman share one "mouth number" (the camera's main face), so the woman (SPEAKER_02) was given to camera 3: 87.9 % of the speech right.

Fix

person_map() computes the lift for every tracked PERSON (camera + track id) instead of every camera. speaker_map.json keeps the same format (speaker -> camera), so nothing changes for the team.

ai_director/perception/speaker.py - person_map() (lines 33-56)

```
33  def person_map(s, persons, min_samples=20):
34      """{"SPEAKER_00": {"cam": 2, "x": 0.77, "lift": 4.7}, ...}: for each speaker, the PERSON (one tracked head on a
35      close-up camera, x = where they sit, 0..1) whose mouth moves MORE THAN USUAL while that speaker talks (the lift of
36      speaker_map, per person: two people on one camera no longer share one mouth number). The camera-level map gave
37      s03's woman (next to the presenter on camera 2) to camera 3; per person: 100 % of the speech right on s01-s03.
38      persons = run.py's rows (t, cam, id, x, found, mouth_motion). Only people whose face was seen for min_samples
39      (2 s) both while the speaker talks and while the others talk; a speaker with nobody like that is left out."""
40      roles = {c["id"]: c["role"] for c in read_json(s / "session.json")["cameras"]}
41      if persons is None or not len(persons):
42          return {}
43      p = persons[(persons.cam.map(roles) == "closeup") & persons.found & persons.mouth_motion.notna()]
44      diar = read_json(s / "diarization.json")
45      who, mm, out = talker(p.t.values, diar), p.mouth_motion.values, {}
46      groups = p.groupby(["cam", "id"]).indices                          # one tracked person -> their rows
47      for spk in sorted({d["speaker"] for d in diar}):
48          mine, others = who == spk, (who != spk) & (who != None)                    # noqa: E711
49          for (cam, _), i in groups.items():
50              on, off = mm[i][mine[i]], mm[i][others[i]]
51              if len(on) < min_samples or len(off) < min_samples:
52                  continue
53              lift = (on.mean() + 1e-3) / (off.mean() + 1e-3)
54              if spk not in out or lift > out[spk]["lift"]:
55                  out[spk] = {"cam": int(cam), "x": round(float(np.median(p.x.values[i])), 2), "lift": round(float(lift), 2
    )}
56      return out
```

Line(s)	What it does, in simple words
40-42	The camera roles; no per-person rows -> nothing to do.
43	Only people on close-up cameras whose face was found and whose mouth movement is known.
44-46	Who talks at every moment (diarization); group the rows by person (camera + track id).
47-48	For every speaker: the moments they talk, and the moments someone ELSE talks.

Line(s)	What it does, in simple words
49-52	For every person: their mouth movement in both; skip people seen for less than 2 s in either.
53-55	Lift = how much more the mouth moves while this speaker talks. Keep the person with the highest lift, with their camera and their usual x position.
56	Speaker -> {camera, x, lift}.

Session	What the log prints (speaker -> camera, where the person sits)
s01	SPEAKER_00 camera 2 x 0.80 (host) · SPEAKER_01 camera 3 x 0.49 (guest) · SPEAKER_02 camera 2 x 0.18 (man in grey)
s02	SPEAKER_00 camera 2 x 0.52 · SPEAKER_01 camera 3 x 0.46 · SPEAKER_02 camera 2 x 0.52 (the same person: extra label)
s03	SPEAKER_00 camera 3 x 0.53 · SPEAKER_01 camera 2 x 0.23 (presenter) · SPEAKER_02 camera 2 x 0.77 (woman) · SPEAKER_03 camera 2 x 0.77 (same as the woman)

What we gained

Speaker map right on 100 % of the speech in all three sessions (s03: 87.9 -> 100 %).

4.5 Follow the talker on a two-person camera

Problem

A camera that shows two people reported the BIGGEST face, often the listener. s01: when the man next to the host talked, camera 2's face was his in only 14 % of the moments; the 9:16 crop went to the host.

Fix

follow_talker(): while a speaker talks and their person sits on that camera, the camera's face numbers become that person's face.

ai_director/perception/speaker.py - follow_talker() (lines 59-77)

```
59  def follow_talker(s, df, persons, people, near=0.25):
60      """A camera that shows two people: while a speaker talks (diarization) whose person (person_map) sits on it, the
61      camera's face numbers become THAT person's face, instead of the biggest face (often the listener, e.g. the man
62      next to s01's host). The scorer then sees the talker's face and the 9:16 crop (face_x) goes to them. Rows where
63      the talker's face wasn't found (or nobody talks) keep the biggest face. near = how far (0..1 of the width) a head
64      may be from the person's usual place."""
65      if not people or persons is None or not len(persons):
66          return
67      p = persons[persons.found]
68      who = talker(p.t.values, read_json(s / "diarization.json"))
69      pick = np.zeros(len(p), bool)
70      for spk, v in people.items():
71          pick |= (who == spk) & (p.cam.values == v["cam"]) & (np.abs(p.x.values - v["x"]) < near)
72      cols = ["face_area", "face_x", "frontalness", "mouth_motion"]
73      chosen = (p[pick].sort_values("face_area").drop_duplicates(["t", "cam"], keep="last")   # one face per camera
74                .set_index(["t", "cam"])[cols].fillna({"mouth_motion": 0.0}))
75      at = pd.MultiIndex.from_arrays([df.t.values, df.cam.values])
76      hit = at.isin(chosen.index)
77      df.loc[hit, cols] = chosen.loc[at[hit], cols].values
```

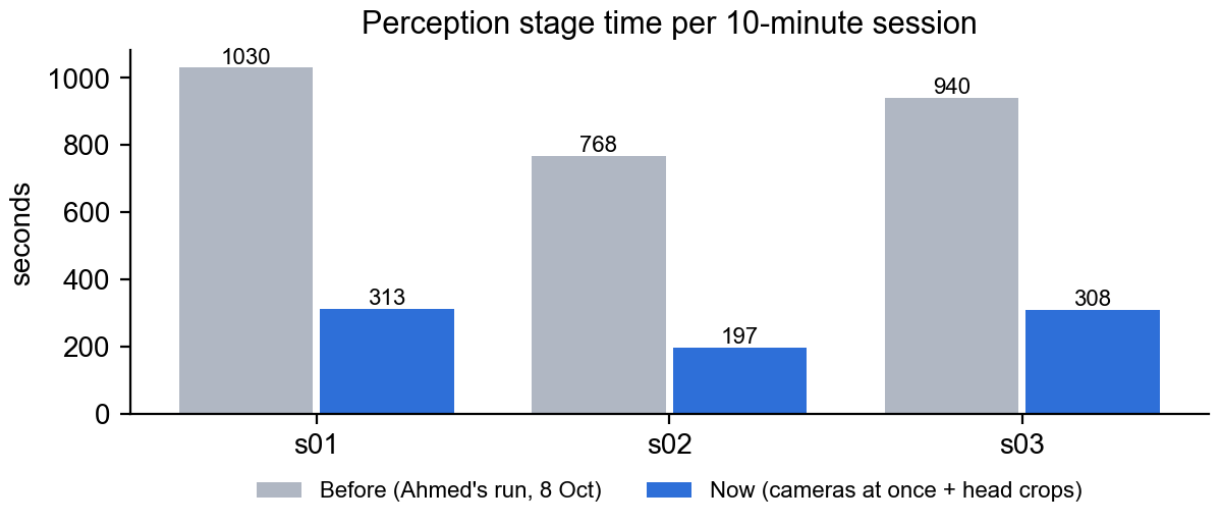
Line(s)	What it does, in simple words
65-68	Nothing to do without the person map; take the rows where a face was found and who talks then.
69-71	Pick a person when: the speaker talking is theirs, on the right camera, and the head is near their usual place (within a quarter of the width).
72-74	One face per camera and moment (the biggest picked one); mouth movement not known yet -> 0.
75-77	Replace the camera's face numbers at exactly those moments.

Speaker talks (camera 2)	Camera 2's face is the talker's: before	now
s01 SPEAKER_02 (man in grey)	14 %	99 %
s03 SPEAKER_01 (presenter)	81 %	96 %
s03 SPEAKER_02 (woman)	34 %	93 %
s03 SPEAKER_03	35 %	100 %

What we gained

The scorer sees the talker's face, and vertical (9:16) clips crop on the person who is talking.

4.6 Perception time on the full sessions



What we gained

1030 -> 313 s (s01), 768 -> 197 s (s02), 940 -> 308 s (s03): 3-4x faster, more faces found, a better map.

Done when (check it yourself)

logs/3_perception.log shows "all cameras done in ..." (about 200-310 s) and one line per speaker with camera and x position; speaker_map.json is s01 {00:2, 01:3, 02:2}, s02 {00:2, 01:3, 02:2}, s03 {00:3, 01:2, 02:2, 03:2}.

5. Director: the face-size rule and the split screen (9 October)

5.1 The face-size rule in the scorer

The scorer gives every camera a score 10 times per second; the speaker part is 3.0 x speaker_prob x **size**, where size grows with the face (full at 15 % of the frame).

Problem

"No face found" counted as a third (no_face_size 0.33), but a small face that WAS found counted only face_area / 0.15 (1.6 % -> 0.11). So finding the faces made s03 camera 2 score LOWER: coverage only 78.2 % instead of the 83 % we expected.

Camera	Faces found before -> now	Median face size	Average "size" before -> now
s02 camera 2	58 -> 100 %	2.2 % of the frame	0.23 -> 0.14
s03 camera 2	22 -> 100 %	1.6 % of the frame	0.28 -> 0.11
s03 camera 3	91 -> 100 %	1.6 % of the frame	0.14 -> 0.11

Fix

If anyone is on the camera, size is never below no_face_size (0.33); a big face still counts more (up to 1). Two lines in director/scorer.py (Ahmed approved). The mock session uses the same rule.

ai_director/director/scorer.py - score(), the size rule

```
28      size = (f.face_area / CFG["face_full"]).clip(upper=1)
29      someone = (f.num_people > 0) | (f.face_area > 0)
30      size = size.clip(lower=CFG["no_face_size"]).where(someone, 0.0)   # someone on camera: at least a third, a big face m
    ore
31      base = (W["speaker"] * f.speaker_prob * size
```

Rule (new perception numbers)	s01	s02	s03
Current rule (area / 0.15, no face = 0.33)	86.6 %	88.5 %	78.2 %
At least 0.33 if someone is there (chosen)	**87.0 %**	**92.1 %**	**85.9 %**
face_full 0.05 (rejected: tuned on all 3 sessions)	87.9 %	91.8 %	85.7 %

What we gained

Speaker coverage 87.0 / 92.1 / 85.9 %; the listener on screen while someone talks: s02 9.0 -> 5.8 %, s03 12.5 -> 3.3 %. On our footage almost every close-up face is under 5 % of the frame, so in practice all close-ups count the same third and the speaker part depends on WHO talks.

5.2 Split screen: when to show both close-ups at once

Problem

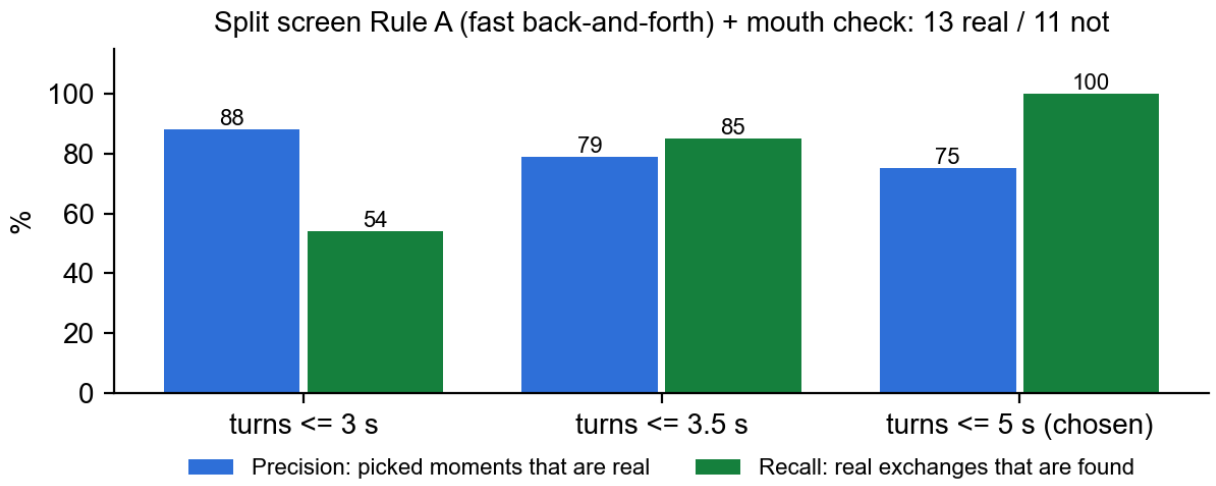
When two people on different close-ups talk quickly in turns, the edit "ping-pongs": it cuts every 2-3 s between camera 2 and camera 3. Ahmed asked for camera 2 + camera 3 side by side SOMETIMES (not always), when two people talk to each other, like a TV director does.

Two rules decide when:

- **Rule A - fast back-and-forth.** Speech on the same close-up camera in a row counts as ONE camera turn (two people on camera 2 are one shot). A moment = at least 3 turns that switch camera every time, each turn at most **T** seconds, pauses at most 2 s, at least 4 s long. Plus the **mouth check**: in most turns the talker's mouth must move more than the other close-up person's (vision, independent of the audio's speaker labels, so it catches wrong labels).
- **Rule B - the listener reacts.** Laughter in the audio (score >= 0.25) or the AI Brain says laughter / reaction, while a person on one of the close-ups talks: show the talker AND the one reacting (+-1 s, at least 4 s).

5.3 How accurate is it? Tested before building

Ahmed asked to see the accuracy first. Every candidate moment on s01-s03 was labelled by hand from its transcript lines, the camera 2 and camera 3 frames and the mouth movement: **13 real fast exchanges** and **11 not needed** (normal interview question-then-answer, one person talking with a 0.3 s interjection, a wrong speaker label). Two words: **precision** = of the moments the rule picks, how many are real (low = splits at the wrong time); **recall** = of the real exchanges, how many it finds (low = missed moments).



Each turn at most	Precision (no mouth check)	Precision + mouth check	Recall	Split time s01 / s02
3 s	78 %	88 % (7/8)	54 %	6 % / 4 %
3.5 s	73 %	79 % (11/14)	85 %	11 % / 9 %
5 s (chosen)	67 %	**75 % (12/16)**	**100 %**	18 % / 13 %
6 s	52 %	-	100 %	25 % / 23 % (no mouth check)

What else the test showed:

- The mouth agrees with the audio's speaker labels on 91-98 % of all camera 2 / camera 3 turns; the check removed 2 wrong moments and no right one.
- Rule B: 4 moments, all in s01 (1:27, 4:54, 4:58, 5:59), **4 of 4 right** by eye (the person on the other close-up laughs or smiles). s02/s03: none, because no laughter was found there (what it misses can't be measured until Ahmed gives the laughter times).
- s03 never gets a split: 3 of its 4 speakers sit on camera 2, so there is no camera 2 / camera 3 exchange.
- s01 camera 2 has TWO people (the host and the man in grey): a half picture centred on the camera's usual face would show the host even when the man in grey talks (3 of 7 moments). Each half must follow the person IN the exchange.
- Most ping-pong is not fast turn-taking: with 5 s turns only 33 of s01's 62 short camera 2 / camera 3 shots (s02: 12 of 37) are inside a split moment.

Ahmed's decision

Turns up to 5 s first (in the config, so it can be changed), plus Rule B; 16:9 only (never in vertical clips); no split in s03 is fine.

5.4 How it is built

Fix

New file **ai_director/director/split.py**: split_moments() finds the moments (Rule A + mouth check, Rule B, both close-ups must show a face, moments less than 3 s apart become one); add_splits() puts them into the switcher's cut list. director/run.py decide() calls them at the end, so the main video, the agent's re-edits and its "mostly the guest" clips all get splits.

Each half = half the width of the camera, centred on the face of THAT camera's person while they talk (face_at; s01 camera 2: the man in grey at x 0.17-0.20 or the host at 0.77-0.82). Camera 3 is on the left and camera 2 on the right (config split.order), so the two people face each other.

director/render.py: a split shot is made from its two cameras with split_filter() (below); it has the same size as every other shot, so all shots still join without re-encoding.

Vertical (9:16) clips never split: a split shot is shown from its main camera. The evaluation's speaker coverage counts a speaker on either half.

ai_director/director/split.py - add_splits() (lines 143-174)

```
143  def add_splits(cuts, moments, min_shot):
144      """The switcher's cut list with every split moment put in: the shots inside become one split shot; a piece of a
145      shot shorter than min_shot left next to a split joins the split (no flash)."""
146      out = [dict(c) for c in cuts]
147      for m in moments:
148          keep = []
149          for c in out:
150              if c["end"] <= m["start"] or c["start"] >= m["end"]:
151                  keep.append(c)
152                  continue
153              if c["start"] < m["start"]:
154                  keep.append({**c, "end": m["start"]})
155              if c["end"] > m["end"]:
156                  keep.append({**c, "start": m["end"]})
157          shot = {k: m[k] for k in ("start", "end", "cam", "split", "split_x")}
158          out = sorted(keep + [shot], key=lambda c: c["start"])
159      for i, c in enumerate(out):                                          # short leftovers join the split
160          if c.get("split"):
161              continue
162          near = [j for j in (i - 1, i + 1) if 0 <= j < len(out) and out[j].get("split")]
163          if near and c["end"] - c["start"] < min_shot:
164              j = near[0]
165              out[j]["start"], out[j]["end"] = min(out[j]["start"], c["start"]), max(out[j]["end"], c["end"])
166              c["gone"] = True
167      out = [c for c in out if not c.get("gone")]
168      merged = []
169      for c in out:                                                        # the same camera twice in a row = 1 shot
170          if merged and not c.get("split") and not merged[-1].get("split") and merged[-1]["cam"] == c["cam"]:
171              merged[-1]["end"] = c["end"]
172          else:
173              merged.append(c)
174      return [{**c, "start": round(c["start"], 2), "end": round(c["end"], 2)} for c in merged]
```

Line(s)	What it does, in simple words
143-145	The function and its description.
146	A copy of the switcher's cut list (so the original is not changed).
147-156	For every split moment: shots outside it are kept; a shot that crosses its start or end is cut there.
157-158	The split shot itself goes in (start, end, main camera, the two cameras, the two face positions); sort by time.
159-167	A left-over piece shorter than min_shot (2 s) next to a split joins the split: no flash.
168-173	Two shots of the same camera in a row become one shot.
174	Times rounded to 0.01 s.

ai_director/director/render.py - split_filter() (lines 15-23)

```
15  def split_filter(i, j, xs, w, h, out):
16      """Split screen (2026-10-09): inputs i and j side by side, each cut to a half-width strip around its person's face
17      (xs = face positions 0..1, None = the middle), a thin dark line between -> one w x h picture called [out]."""
18      cw = w // 4 * 2                                  # half the width, an even number of pixels
19      x = [int(min(max((0.5 if fx is None else fx) * w - cw / 2, 0), w - cw)) for fx in xs]
20      line = SPLIT["line"]
21      return (f"[{i}:v]crop={cw}:ih:{x[0]}:0,scale={cw}:{h},setsar=1[l{out}];"
22              f"[{j}:v]crop={cw}:ih:{x[1]}:0,scale={cw}:{h},setsar=1[r{out}];"
23              f"[l{out}][r{out}]hstack=inputs=2,drawbox=x={cw - line // 2}:y=0:w={line}:h={h}:color=black:t=fill[{out}]")
```

Line(s)	What it does, in simple words
15-17	The function and its description: two camera inputs -> one picture.
18	Half the width, as an even number of pixels (video encoders need even sizes): 1920 -> 960.
19	Where each half starts: centred on the person's face, but never outside the picture (None = the middle).
20	The width of the dark line between the halves (config split.line, 4 px).
21-23	The FFmpeg filter: cut each camera to its half, make both the same size, put them side by side (hstack) and draw the line in the middle.

Contract change (tell Ahmed Sameh)

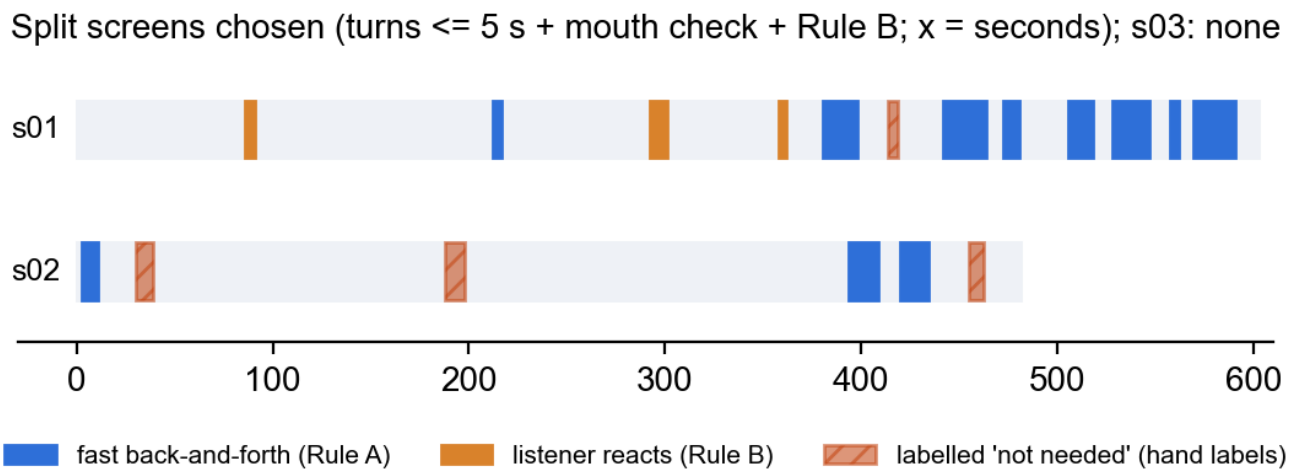
A split shot in cuts.json has two new OPTIONAL fields:

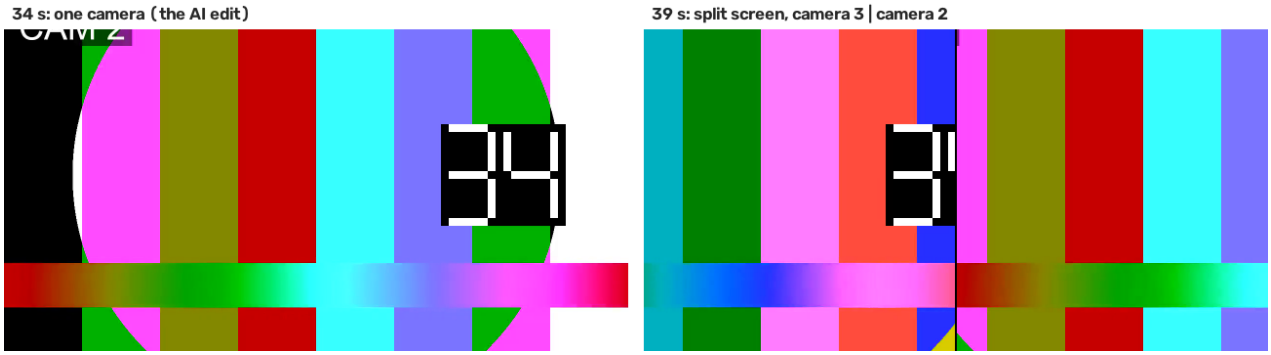
`{"start": 441.5, "end": 465.2, "cam": 2, "split": [3, 2], "split_x": [0.52, 0.703]}` (real, s01)

split = [left camera, right camera]; split_x = where each face is (0..1); cam = the camera of the person who talks most, so older code that only reads cam still works. The full code of split.py is in Appendix D.

5.5 Results

Same data (Ahmed's 9 Oct run)	s01	s02	s03
Split screens / time in split	12 / 138 s (23 %)	6 / 70 s (15 %)	0
Speaker coverage without -> with split	87.0 -> **91.3 %**	92.1 -> **95.0 %**	85.8 % (same)
Shots in the main video	125 -> 93	90 -> 76	83 (same)
Average shot	4.8 -> 6.5 s	5.4 -> 6.3 s	7.3 s
Shots shorter than 2 s	0	0	0





Above: the mock session (a test pattern) rendered with its split at 37-41 s. The real s01 program.mp4 was rendered on a copy (2 minutes, full length, 1920x1080) and checked frame by frame: camera 3's guest on the left and the camera 2 person on the right face each other, the right camera 2 person is chosen every time, and the captions stay readable across both halves. (Real frames are not printed here: no real people in shared PDFs.)

What we gained

More of the talking time shows the talker (s01 +4.3, s02 +2.9 points), fewer and longer shots, and the listener's reaction is visible in fast exchanges and laughs. 9 new tests in tests/test_split.py.

Done when (check it yourself)

Re-run Switching + Render (part 11): the director prints "12 split screens, 138 s" for s01 and "6 split screens, 70 s" for s02, and program.mp4 shows two people side by side at about 7:30 in s01.

To use fewer, surer splits set split.turn_max: 3.5 in configs/default.yaml; split.enabled: false turns them off.

6. AI Brain (8-9 October)

Every 5 seconds the AI Brain sends a 2x2 picture of all cameras + what was said to a local vision model, which answers a short JSON: a sentence about the scene, the event, the energy, and the editing **mode** (conversation, reaction, wide_energy, demo). Modes change the camera multipliers (configs/modes.yaml); demo, for example, gives the wide camera x1.3 and the close-ups x0.9.

6.1 The prompt and laughter from the audio (8 October)

Problem

The old prompt ("choose the best editing style") answered "conversation" for 337 of 339 pictures.

Fix

A new prompt that describes what each mode LOOKS like, 2x bigger camera tiles (640x360), and laughter taken from the audio (3.3).

What we gained

A test version got 19/20 on s03 test moments.

6.2 Why the AI Brain took 22-41 minutes (9 October)

Problem

13.4 s (s01), 13.8 s (s02) and 20.5 s (s03) per moment x 121 moments = 22-41 minutes per session.

The answer itself is ~40 tokens of JSON, but the installed **qwen3-vl:4b** is the THINKING version (in Ollama it is the same file as qwen3-vl:4b-thinking): it writes 550-2,800 hidden "reasoning" tokens first (2,550-6,574 characters). Ollama 0.35.1 ignored think=False and "/no_think"; some answers even filled the whole 4,096-token context.

One moment (s03)	Time
Building the 4-camera picture	0.2-0.9 s
The model reading the picture and the prompt (~1,300 tokens)	0.7 s
The model writing its answer (548-2,786 tokens!)	7-39 s

6.3 How accurate is it? A hand-labelled test

We picked 26 moments (on purpose with many "demo" answers), looked at the same pictures the model sees and wrote down the right mode BEFORE reading the model's answers. Then we compared.

Problem

Thinking model: 17 of 26 right. Conversation 13/13, reaction 1/1, but **demo only 2 of 11**.

s03: a laptop just lying on the table while people talk became "demonstrates a laptop".

s02: there is NO laptop at all, but people TALK about an app ("breast detection", "supervision features"), so it said "demo": it decided from the words, not the picture.

A stricter prompt alone: 18/26 (fixed 4 false demos, made 3 new ones).

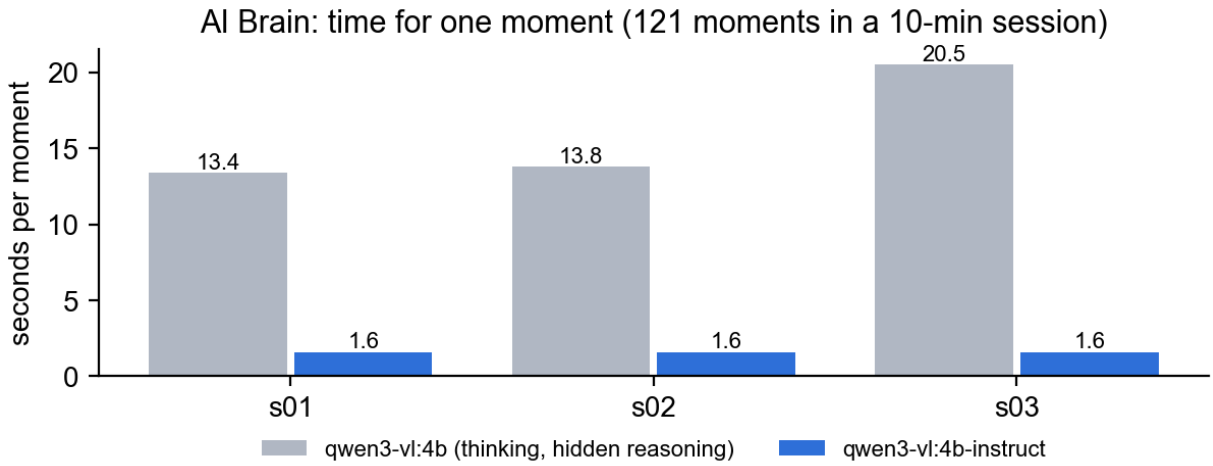
6.4 The instruct model + a stricter demo rule

Fix

config ollama.vision_model: **qwen3-vl:4b-instruct** (3.3 GB; ollama pull qwen3-vl:4b-instruct).

brain/scene.py PROMPT: demo = someone is touching, pointing at or turning a laptop / screen / object towards the others, or holds an object up; a laptop lying on the table is NOT a demo, people only talking about an app is NOT a demo; the words are only there to understand the talk.

Test	Thinking + old prompt	Instruct + old prompt	Instruct + stricter prompt (chosen)
26 hand-labelled moments	17/26	24/26 (never says demo)	24/26 (both real demos found)
False demo in s01 + s02 (all 218 moments)	3	0	0
s03 demo answers that were right (checked by eye)	about 2 of 11	5 of 7 (misses most)	about 12 of 20
Reaction answers that were right	-	1 of 4	1 of 1
Seconds per moment	13-20	1.0-2.0	1.0-2.0



ai_director/brain/scene.py - the prompt

```
PROMPT = """You are a TV director choosing the editing style for this moment of a multi-camera show. The image shows the
cameras (CAM {wide} is the wide shot of everyone).
What people said in the last {window} seconds (only to understand the talk; decide the style from what you SEE):
"{text}"
Look at what the people DO:
- demo: someone is touching, pointing at, or turning a laptop, a screen or an object towards the others, or holds an
  object up. A laptop or screen that just lies on the table while people talk is NOT a demo, and people only TALKING
  about an app or a product is NOT a demo.
- wide_energy: several people move a lot at the same time
- reaction: someone laughs or is clearly surprised
- conversation: people talk normally (sitting or standing)
Return JSON:
scene: one short English sentence about what the people do;
event: talking (someone speaks), laughter, demo, standing, silence (nobody speaks), other;
energy from 0 (calm) to 1 (very lively);
mode: one of the four styles above;
highlight: true only if this moment is memorable."""
```

What we gained

The real stage on s03: **254 s instead of 2,484 s** (about 10x faster). The model said demo 19 times, 11 stayed after smoothing (a mode needs the same answer twice in a row); 7 of those 11 are real demos (1:40-1:50, 3:30-3:35, 6:35-6:40) instead of 29 mostly wrong ones.

Tell Eslam: his prompt changed and the plain qwen3-vl:4b must not be used.

Done when (check it yourself)

logs/4_ai.log starts with "qwen3-vl:4b-instruct, one picture every 5 s" and the AI Brain line in logs/times.json is about 200-260 s.

7. Render, video quality and highlights

7.1 1080p everywhere (8 October)

Problem

The synced camera copies, the main video, the highlights and the clips were 720p.

Fix

configs ingest.render_height: 1080; python -m ai_director.ingest.proxy data/sessions/s01 remakes the copies with the SAME sync offsets (audio.wav is kept, tested: the same file, md5).

render_clip's 9:16 crop follows the source: 1080p -> a 608 px strip -> 1080x1920.

render_clip(source=[shots]) cuts every shot from its own camera in ONE FFmpeg run (the agent's "mostly the guest" clips).

What we gained

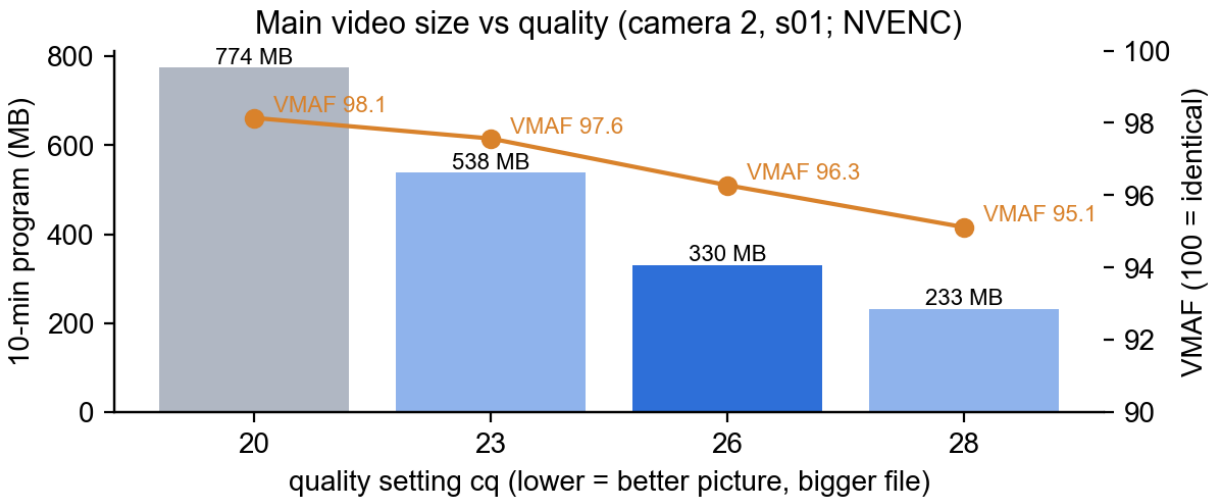
Full HD main video, highlights and clips.

7.2 Video quality in the config, main video cq 26 (9 October)

Problem

A 10-minute program.mp4 was 716 MB (quality cq 20) - heavy for the web app and phones.

We encoded 30 s of camera 2 and of the wide camera exactly like the render stage and measured the size and **VMAF** (Netflix's "does it look the same" score, 100 = identical; above ~95 viewers can't see a difference). A face crop at cq 20 / 26 / 28 looked the same.



cq	10-min program (camera 2 / wide)	VMAF (camera 2 / wide)
20 (before)	774 / 557 MB	98.1 / 97.6
23	538 / 394 MB (-30 %)	97.6 / 97.2
26 (chosen)	**330 / 224 MB (-58 %)**	**96.3 / 95.9**
28	233 / 158 MB (-70 %)	95.1 / 94.9
26, slowest preset p7	the same as 26	the same (no gain)

Fix

All quality numbers are now in configs/default.yaml -> video.quality (program 26, clips 21, cameras 22; the "proxy" entry went away with the 640 px copies, part 7.5) and read by common/io.quality(); render_program, burn_captions, render_clip and the ingest copies use it.

ai_director/common/io.py - quality() (lines 50-57)

```
50  @lru_cache(maxsize=None)
51  def quality(kind):
52      """The H.264 quality level for one kind of video ("program", "clips", "cameras"), from
53      configs/default.yaml -> video.quality (lower = better picture, bigger file)."""
54      import yaml
55      from .paths import CONFIGS
56      cfg = (yaml.safe_load((CONFIGS / "default.yaml").read_text(encoding="utf-8")).get("video") or {}).get("quality")
57      return int((cfg or {}).get(kind, QUALITY[kind]))
```

What we gained

About 58 % smaller main videos with no visible difference; you can change the numbers any time in the config.

7.3 Highlight titles with Gemini (9 October)

Fix

highlights.title_model: gemini-3.1-flash-lite picks the moments and writes the titles (1 request per session); qwen3:8b only if Gemini can't be used. Both use the shared helper gemini.ask_json().

ai_director/highlights/select.py - default_ask() (lines 66-80)

```
66  def default_ask(prompt):
67      """Gemini (highlights.title_model, 1 request per session) writes the titles; the local model (ollama.text_model)
68      only if Gemini can't be used (no key / daily limit / no internet). 2026-10-09 (Ahmed): qwen3:8b's titles were weak
69      and Gemini translated much better on s01 + s03."""
70      if CFG.get("title_model"):
71          try:
72              from ..audio.gemini import ask_json
73              return ask_json(CFG["title_model"], prompt, Picks)
74          except Exception as e:
75              print(f"  Gemini titles failed ({type(e).__name__}: {str(e)[:120]}) -> the local model")
76      from ..brain.llm_client import chat_json                    # Eslam's client (needs the ollama package)
77      from ..common.paths import CONFIGS
78      import yaml
79      model = yaml.safe_load((CONFIGS / "default.yaml").read_text(encoding="utf-8"))["ollama"]["text_model"]
80      return chat_json(model, prompt, Picks, think=False)
```

7.4 Vertical clips: the strip moves to whoever talks (9 October)

Ahmed asked: when a vertical (9:16) clip is made and the face moves, does the picture follow it? **No**: a 9:16 clip is cut at ONE position for the whole clip (the main talker's usual face position; one per camera when a clip mixes cameras). We measured whether that is a problem (every 30 s piece of s01-s03 + the real highlights):

- **People don't move out of the strip.** With one person on the camera the face is outside the strip 0-0.2 % of the time (s03's standing presenter only moves between x 0.18 and 0.30).
- **The real gap is a camera with TWO people** (s01 camera 2, s03 camera 2). The strip is only 32 % of the picture's width, so it can't hold both; it stayed on the main talker, and the other person was never seen talking (s01 highlight h3: the man in grey talks 11.5 of its 37 s, always outside the strip).

Fix

highlights/clips.py **people_x()**: where each person sits on their camera = the camera's face position while THAT person talks, over the whole session (s01 camera 2: host 0.77, man in grey 0.19).

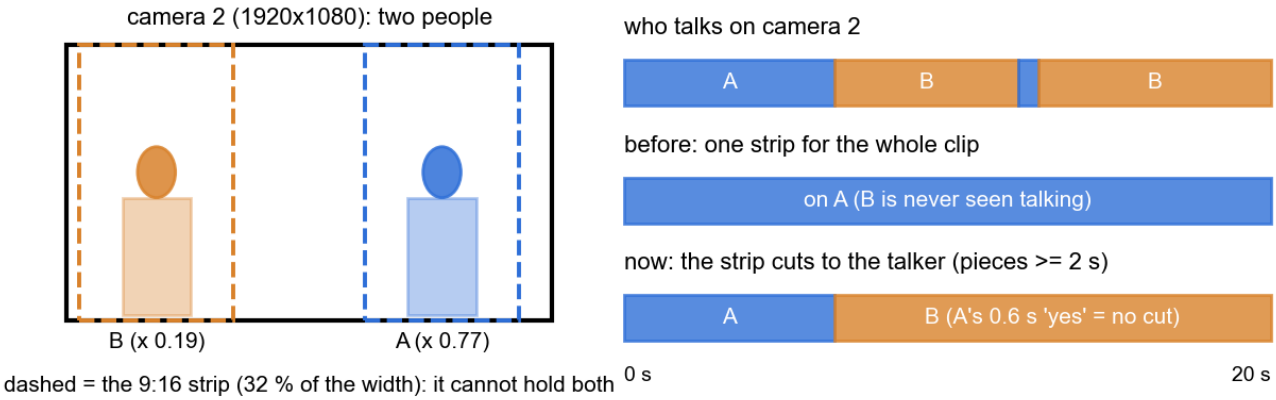
strip_shots() (below): on a camera whose people are more than 0.25 apart, the strip cuts to whoever on that camera is talking, like a cut between two close-ups. Every piece lasts at least 2 s (the switcher's min_shot), so a quick "yes" makes no cut; when nobody on that camera talks, it stays.

Used by the highlights' 9:16 clips and the agent's 9:16 clips; when the user asks the agent for a person ("vertical clip of the host"), the strip stays on THAT person. render_clip: a shot's own crop_x wins.

ai_director/highlights/clips.py - strip_shots() (lines 33-75)

```
33  def strip_shots(s, shots, feats, keep=None):
34      """9:16 on a camera with two people too far apart for one strip (2026-10-09): the strip moves to the one who
35      talks, like a cut between two close-ups (each piece >= switcher.min_shot s, so a quick "yes" is no flash; nobody on
36      that camera talks -> it stays). Other cameras: their face in the shot. keep = {cam: x}: always there (the person the
37      user asked for). shots [{start, end, cam, ...}] -> the same with crop_x per shot (None = the middle)."""
38      pos, diar = people_x(s, feats), read_json(s / "diarization.json") if (s / "diarization.json").exists() else []
39      out = []
40      for sh in shots:
41          cam, a, b = sh["cam"], sh["start"], sh["end"]
42          who = pos.get(cam, {})
43          if keep and cam in keep:
44              out.append({**sh, "crop_x": keep[cam]})
45              continue
46          if len(who) < 2 or max(who.values()) - min(who.values()) < FIT:     # one person, or both fit in the strip
47              x = feats[(feats.cam == cam) & feats.t.between(a, b) & (feats.face_area > 0)].face_x
48              out.append({**sh, "crop_x": round(float(x.median()), 3) if len(x) else None})
49              continue
50          talks = sorted((max(d["start"], a), d["speaker"]) for d in diar
51                         if d["speaker"] in who and d["end"] > a and d["start"] < b)
52          if talks:
53              first = talks[0][1]
54          else:                                                               # nobody there talks: whoever is seen
55              x = feats[(feats.cam == cam) & feats.t.between(a, b) & (feats.face_area > 0)].face_x
56              first = min(who, key=lambda p: abs(who[p] - (x.median() if len(x) else 0.5)))
57          pieces = [[a, first]]                                               # [start, person]
58          for t, spk in talks:
59              if spk != pieces[-1][1]:
60                  pieces.append([t, spk])
61          changed = True
62          while changed and len(pieces) > 1:                                  # a piece < min_shot joins the one before
63              changed = False
64              ends = [p[0] for p in pieces[1:]] + [b]
65              for i, (p, e) in enumerate(zip(pieces, ends)):
66                  if e - p[0] < RULES["min_shot"]:
67                      if i == 0:
68                          pieces[1][0] = a                                    # the first one: the next starts at a
69                      pieces.pop(i)
70                      changed = True
71                      break
72              pieces = [p for k, p in enumerate(pieces) if k == 0 or p[1] != pieces[k - 1][1]]   # same person twice
73          ends = [p[0] for p in pieces[1:]] + [b]
74          out += [{**sh, "start": round(p[0], 3), "end": round(e, 3), "crop_x": who[p[1]]} for p, e in zip(pieces, ends)]
75      return out
```

Line(s)	What it does, in simple words
33-37	The function and its description: shots in, the same shots with a crop position each out.
38	Where each person sits (people_x) and who talks when (diarization.json).
40-42	For every shot: its camera, start, end, and the people on that camera.
43-45	The user asked for a person on this camera -> the strip stays on them.
46-49	One person, or two who fit in one strip (less than 0.25 apart) -> one strip on the face, as before.
50-56	Who on this camera talks in the shot, in time order; the strip starts on the first one (nobody talks -> the person whose face is seen).
57-60	A new piece every time a DIFFERENT person on this camera starts talking.
61-72	A piece shorter than 2 s joins the one before (a quick "yes" = no cut); the same person twice in a row becomes one piece.
73-75	The pieces become shots, each cropped on its person's position.



s01: talker outside the strip 16.8 % -> 0.9 % of the time; s03: 11.3 % -> 0.5 %

Every 30 s piece + the highlights	s01	s02	s03
Talker on the strip's camera but OUTSIDE the strip: before	16.8 %	0 %	11.3 %
... now	**0.9 %**	0 %	**0.5 %**
Strip cuts per 30 s clip	0.4	0 (its two cam-2 labels sit at the same spot)	0.6

What we gained

The person talking is in the vertical clip. Checked on two real highlights rendered on a copy: s01 h3 shows the host, then cuts to the man in grey at 214.9 s as he starts talking; s03 h2 cuts from the man in the blue shirt to the woman at 30.4 s. 5 new tests in tests/test_strip.py.

Done when (check it yourself)

Re-run the Highlights stage (part 11): the log line of a two-person clip says "9:16 from 2 shots" (or more) instead of one camera.

7.5 The 640 px copies removed (9 October)

Problem

Ingest made a small 640 px copy of every camera (video/camN_640.mp4) for perception and the AI Brain (the first guides). Since 8-9 October both read the 1080p video and shrink the frames themselves, so the copies cost time (about 19 s per camera) and space (330 MB for s01-s03) for nothing.

Ahmed asked to delete them **only if nothing uses them**. Checked first:

Who could read it	Uses the 640 px copy?	Done
Perception (Sara)	No: reads video/camN.mp4	-
AI Brain (Eslam)	No: reads video/camN.mp4	-
paths.proxy() helper	Never called anywhere	Removed
perception/debug_video.py	YES (the only reader)	Reads camN.mp4 and shrinks to 640 px itself (tested)
common/check.py (contract check)	Required the file	No longer required
Web app (Ahmed Sameh's guide)	Never mentions it	-
Tests + the mock maker	2 tests used the mock's copies	Use camN.mp4; the mock no longer has them
ingest/proxy.py + configs	Made them (ingest.proxy_width, video.quality.proxy)	Removed

What we gained

Faster ingest and 330 MB less. The 12 files (s01-s03 + the mock) went to the Windows **Recycle Bin**, so they can be restored; empty it to free the space. All three sessions still pass the contract check.

Contract change: video/camN_640.mp4 no longer exists (tell the team). The mock's 3 files are in Git, so they show as deleted when you commit.

8. Every new or changed setting (configs/default.yaml)

Setting	Value	What it does
ingest.render_height	1080	Size of the synced camera copies (was 720)
ingest.proxy_width	(removed)	There are no 640 px copies any more (part 7.5)
video.quality.program / clips / cameras	26 / 21 / 22	H.264 quality per kind of video (lower = better picture, bigger file); program was 20; the proxy entry was removed
split.enabled / split.order	true / [3, 2]	Split screen on; left camera, right camera (part 5.2-5.5)
split.turn_max / min_turns / min_seconds / max_gap	5.0 / 3 / 4.0 / 2.0	Rule A: each turn at most 5 s, at least 3 turns, at least 4 s long, pauses at most 2 s (3.5 = fewer, surer splits)
split.mouth_check	true	Drop a moment when the mouths disagree with the speaker labels
split.laugh / split.react_pad	0.25 / 1.0	Rule B: audio laughter needed; seconds added before and after
split.join_gap / split.line	3.0 / 4	Splits closer than 3 s become one; px of the dark middle line
agent.timeout	120	Seconds one Ollama request may take, then a "took too long" reply
agent.topic_drop / agent.topic_max	0.03 / 90	A found piece grows over neighbours at most 0.03 less similar, up to 90 s (part 2.5)
perception.workers	3	Cameras processed at the same time (1 = one after another)
perception.face_roles	[closeup]	Cameras whose faces are measured (not the wide camera)
scorer.no_face_size	0.33	Now: the LEAST a person on camera counts (found small face or no face found)
audio.merge_voice	0.95	How similar two labels' voices must be to be joined
audio.english_words_model	gemini-3.5-flash	Puts back the English words (Arabic sessions; null = off)
audio.english_words_fallback	gemini-3.1-flash-lite	Used when 3.5-flash has no quota left
audio.english_words_window	60	Seconds of audio per request
audio.translate_model	gemini-3.1-flash-lite	Translation (qwen3:8b only if Gemini can't be used; null = qwen only)
ollama.vision_model	qwen3-vl:4b-instruct	AI Brain (NOT the plain qwen3-vl:4b = thinking)
brain.tile	[640, 360]	Size of one camera in the brain's picture
brain.laugh / brain.laugh_agree	0.25 / 0.1	Audio laughter -> reaction / needed to trust the vision model's laughter
highlights.title_model	gemini-3.1-flash-lite	Picks the highlights and writes their titles
agent.focus_share	only 100, mostly 75, more 60	Share of the clip on the asked person's camera
agent.boosts	1.2 ... 3.0	Camera boosts tried to reach that share

9. Contract changes and what each teammate needs to do

Who	What changed for you	What to do
Everyone	Translation, English words and highlight titles use Gemini; the AI Brain uses a new model; split screen in the main video; no more video/camN_640.mp4	Put your Gemini keys in .env (GEMINI_API_KEY, GEMINI_API_KEY_2, ...; never in code or Git); ollama pull qwen3-vl:4b-instruct; git pull; run the tests (150 pass)
Eslam (AI Brain, Agent)	vision_model = qwen3-vl:4b-instruct; the demo rule in brain/scene.py PROMPT; laughter from audio in modes.py; agent: asks.py, focus reader, seconds/captions arguments; 9 Oct: the code writes the clip list, number of clips, names, follow-ups, time limit, search >= 5 moments that grow over their topic, unclear-request guard	Read parts 2 and 6; don't switch back to the plain qwen3-vl:4b; keep the rule "the LLM suggests, the code checks" (2.6)
Ahmed Sameh (web app)	clips.json items have "captions"; cuts.json shots may have "split" + "split_x" (part 5.4); video/camN_640.mp4 is gone; the main video is ~300 MB; all outputs 1080p	Show / pass the captions choice; if you draw the cut list, show split shots (or just use "cam"); nothing else breaks
Sara (perception, highlights)	Head crops, per-person speaker map, follow the talker, cameras at the same time; highlight titles from Gemini; 9:16 highlights switch between two people on one camera; debug_video.py reads camN.mp4	Read parts 4 and 7.4-7.5; features.parquet and speaker_map.json keep the same format
Nada / audio	English words step, Gemini translation, captions one at a time, laughter classes, voices merge	Read part 3; transcript.json keeps the same format

10. Tests

Command: `.venv\Scripts\python.exe -m pytest -q` -> **150 passed** (109 on 8 October, 126 on the morning of 9 October). Every Gemini test uses a FAKE Gemini and every agent test a FAKE LLM, so the tests never use your quota and work without internet or GPU. The real-model agent test is separate: `python -m eval.agent_eval --no-render` (part 2.4).

File	New tests (what they check)
test_agent.py	what the user asked (length, language, captions) · the code makes the asked length · captions from the request · bad titles written again · wrong-language answer asked again · who-to-show reader · "mostly the guest" boost · only / nobody · 9 Oct: the code writes the clip list · one clip unless more are asked · names or roles (blank names, Arabic replies) · follow-ups use the earlier clip · a stuck model / a timeout · the code reads the format · the search returns >= 5 moments · a vague re-edit asks a question · a found piece grows over the same topic
test_split.py (new)	fast back-and-forth found · the mouth check drops wrong labels · laughter while someone talks · the split goes into the cut list · the mock's split at its laugh · rendered side by side with the line · a 9:16 clip never cuts a split in half · speaker coverage counts both halves · the contract allows split shots
test_strip.py (new)	where each person sits · the strip moves to the talker (no cut for a 0.6 s "yes") · it stays when a person is asked or both fit · vertical highlights follow the talker · two strips from one camera rendered
test_contracts.py / test_brain.py	no 640 px copies in the mock; the brain's grid reads camN.mp4
test_eval.py	numbers updated: the mock now has one split (37-41 s at its laugh)
test_asr.py	extra voices join only the same person
test_brain.py	laughter from the audio
test_perception.py	lift map + transcript · pose gesture (hands, not walking) · head box + square cut · faces per person · person map with two people on one camera · follow the talker · 3 cameras at once = the same numbers
test_scorer.py	a person without a found face (and a small found face) still counts a third
test_render.py	video quality from the config · a clip from shots · 1080p clips
test_translate.py	Gemini first, then the local model when it stops · Gemini only leaves the GPU alone
test_english_words.py (new)	windows never cut a row · the safety check · English words put back · a window hears the lines around it · a second run is free · slid or empty answers keep the draft · English sessions left alone · no quota -> the fallback model
test_highlights.py	titles from Gemini, then the local model

11. How to re-run the sessions (step by step)

Do this after adding your Gemini keys. Open cmd in the repo folder with the .venv active. Run the commands of each step one after another (or join them on one line with &&).

Step 1 - English words (Arabic sessions only; about 10 gemini-3.5-flash requests each):

```
python -m ai_director.audio.english_words data/sessions/s01
python -m ai_director.audio.english_words data/sessions/s03
```

Step 2 - translate again with Gemini (all sessions; about 20-26 requests each):

```
python -m ai_director.audio.translate data/sessions/s01 --again
python -m ai_director.audio.translate data/sessions/s02 --again
python -m ai_director.audio.translate data/sessions/s03 --again
```

Step 3 - delete the old outputs (keeps the transcript and speakers; scene.partial.jsonl must go, or the brain would reuse the old model's answers):

```
for %s in (s01 s02 s03) do del data\sessions\%s\audio_events.json data\sessions\%s\features.parquet
  data\sessions\%s\speaker_map.json data\sessions\%s\scene.partial.jsonl data\sessions\%s\scene.jsonl
  data\sessions\%s\scores.parquet data\sessions\%s\timeline.json data\sessions\%s\cuts.json
  data\sessions\%s\program_nocaps.mp4 data\sessions\%s\program.mp4 data\sessions\%s\highlights.json
  data\sessions\%s\metrics.json        (type it as ONE line)
```

Step 4 - the pipeline:

```
python -m ai_director.pipeline data/sessions/s01
python -m ai_director.pipeline data/sessions/s02
python -m ai_director.pipeline data/sessions/s03
```

> **Done when (check it yourself)**
>
> Each session's logs/times.json: Perception about 200-310 s, AI Brain about 200-260 s; program.mp4 about 220-330 MB; speaker_map.json as in part 4.6.
>
> Watch at least 2 minutes of every program.mp4: cuts, captions (Arabic + English words in the right order), laughter reactions, s03 demo moments, and the split screens in s01/s02 (part 5.5).

Only the split screen and the switching strip (no Gemini needed): if you don't want to wait for the Gemini keys, these three stages are enough (about 2 min each; Highlights uses 1 Gemini request for the titles, or qwen3:8b without keys). s03 has no splits, but its vertical highlights switch:

```
python -m ai_director.director.run data/sessions/s01
python -m ai_director.director.render data/sessions/s01
python -m ai_director.highlights.run data/sessions/s01
```

Then the same for s02 and s03. Ingest does NOT need to be run again: the 640 px copies are simply gone.

12. How to commit this work

Nothing below is committed yet. Suggested: check the re-run first, then commit in a few clear pieces (commit messages like `ahmed: <what>`, as in the project rules). data/ is never committed (.gitignore). Decide with the team whether CLAUDE.md goes into Git.

```
git checkout -b ahmed/oct-8-9-fixes
git add ai_director/agent tests/test_agent.py ai_director/common/schemas.py samples/make_mock.py samples/mock_s00/clips.json
git commit -m "ahmed: editor agent reads the request, writes the clip list, follow-ups, topic search, time limit"
git add ai_director/perception tests/test_perception.py
git commit -m "ahmed: perception head crops, per-person speaker map, follow the talker, cameras at once"
git add ai_director/audio tests/test_asr.py tests/test_translate.py tests/test_english_words.py
git commit -m "ahmed: English words, Gemini translation, one caption at a time, laughter, voices merge"
git add ai_director/brain tests/test_brain.py
git commit -m "ahmed: AI brain on qwen3-vl:4b-instruct, stricter demo rule, laughter from audio"
git add ai_director/director ai_director/highlights eval/director_eval.py tests/test_split.py tests/test_strip.py
git add tests/test_eval.py tests/test_scorer.py tests/test_render.py tests/test_highlights.py
git commit -m "ahmed: split screen, switching 9:16 strip, scorer face floor, Gemini highlight titles"
git add -A ai_director/ingest ai_director/common configs samples/mock_s00 tests/test_contracts.py
git commit -m "ahmed: 1080p, video quality in the config, no more 640px copies"
git add reports/2026-10-09_Work_Explained.pdf
git commit -m "ahmed: report of the 8-9 October work"
git push -u origin ahmed/oct-8-9-fixes
```

`git add -A` also records the 3 deleted 640 px files of the mock. The pieces are only for reading the history: some middle commits use code from a later one, so run the tests after the last commit (150 pass).

> **Done when (check it yourself)**
>
> git status shows nothing left to commit except files you chose to keep out (for example CLAUDE.md), and the branch is on GitHub.

13. Not done yet, and next ideas

Item	Why it matters
Fill eval/wer_set.json (it still says PASTE HAND-CORRECTED TEXT HERE)	Without typed references we can't PROVE the transcription and the English-words fix are good (even 3 minutes of s03 helps)
Grow the 26 hand-labelled brain moments to ~60 and keep them in eval/	Every brain change gets a real score
Re-run eval/director_eval.py + the results table on the new outputs	Before/after numbers for the report
Report honestly: the 9 Oct changes were tested on all three sessions	The plan says tune on s01+s02 and report on s03; a 4th recording would be a truly unseen test
The A/B viewer test (Days 13-14)	The strongest proof for the presentation
Pre-flight check (free disk, Ollama, models, Gemini keys)	Today's full disk only showed up as a stuck download
Laughter timestamps for s02/s03 (Ahmed)	The detector hears no laughter there - is that right? It also decides Rule B of the split screen
Ahmed checks the split-screen labels by watching (s01 3:32, 7:27, 7:39, 7:53, 8:26, 8:56, 9:18, 9:29; doubtful 6:54; s02 0:03, 6:39, 7:05; normal Q&A 0:30, 7:35)	The accuracy in part 5.3 uses Claude's labels from text and pictures, without hearing the audio
Agent tests 4 and 14 are right only partly (about a quarter of the clip on the expected part)	Better search (e.g. words + meaning together) or more test requests
eval/agent_eval checks only that a clip overlaps the right moment	Also check length, language, captions and who is shown
s02 camera 2: SPEAKER_00 and SPEAKER_02 sit at the same spot (0.53 / 0.57)	Probably one person with two labels (the voice merge needs 95 %)
Empty the Recycle Bin (the 640 px copies, 330 MB); ollama rm qwen3-vl:4b; back up data/sessions to Drive; revoke the Deepgram key pasted in chat on 4 Oct	Disk space (the C: drive is 97 % full), safety

14. Glossary

Word	Meaning
Pipeline / stage	The chain of steps that turns raw recordings into the edited video; each step is a stage
Done file	The file a stage writes LAST; if it exists, the pipeline skips that stage
Contract	The agreed names and formats of the files in a session folder (Architecture PDF section 4)
Diarization	Who speaks when (SPEAKER_00, SPEAKER_01, ...)
Speaker map	Which camera shows which speaker (speaker_map.json)
Lift	How much MORE someone's mouth moves while a speaker talks than while others talk
Head crop	A square cut around one person's head from the full frame, enlarged before the face model looks
Track id	A number YOLO gives each person so the same person keeps the same number from frame to frame
YOLO-pose	A model that finds people and their body points (nose, eyes, wrists, ...)
MediaPipe Face Landmarker	A model that finds a face and 478 points on it (mouth, eyes, nose)
Speaker coverage	Share of the talking time when the talker's camera is on screen
Scorer / switcher	The scorer gives every camera a score 10x per second; the switcher turns scores into cuts
Mode	The editing style the AI Brain suggests: conversation, reaction, wide_energy, demo
Ollama	The program that runs local AI models on the PC (qwen3:8b, qwen3-vl, bge-m3)

Word	Meaning
Thinking / instruct model	A thinking model writes hidden reasoning before answering (slow); an instruct model answers directly
Token	A piece of a word; models read and write tokens (speed = tokens per second)
Prompt	The instructions + data sent to a model
Gemini / quota / free tier	Google's cloud models; the free tier allows a limited number of requests per day per project (the quota)
Fallback	A second option used automatically when the first one fails
Cache	A saved answer that is reused instead of asking again
Window	A ~1-minute piece of the transcript + its audio sent in one request
RTL / RLM	Right-to-left text / an invisible mark that makes a line right-to-left
NVENC / cq	The NVIDIA GPU video encoder / its quality number (lower = better and bigger)
VMAF	Netflix's score of how close a video looks to the original (100 = identical)
Process / parallel	A separate running program; parallel = several at the same time
Scratch run	Running new code on a COPY of a session in a temporary folder
Split screen	Two cameras side by side in one picture (here: camera 3 left, camera 2 right)
Ping-pong	Cutting back and forth between two cameras every few seconds
Precision / recall	Of the moments a rule picks, how many are right / of the right moments, how many it finds
9:16 strip	The tall piece cut out of a wide camera picture for a vertical clip (32 % of its width)
Embedding / similarity	A list of numbers that describes the meaning of a text; two texts about the same topic have a high similarity (the search's score)
Recycle Bin	Windows' bin: deleted files stay there (and can be restored) until it is emptied

Appendix A. Files changed since commit 4a97784

File	Lines added	Lines removed
ai_director/agent/agent.py	90	25
ai_director/agent/prompts.py	53	13
ai_director/agent/search.py	35	5
ai_director/agent/tools.py	221	38
ai_director/audio/asr.py	6	3
ai_director/audio/events.py	14	3
ai_director/audio/gemini.py	13	1
ai_director/audio/run.py	12	6
ai_director/audio/subtitles.py	88	16
ai_director/audio/translate.py	81	16
ai_director/audio/voices.py	58	0
ai_director/brain/modes.py	25	5
ai_director/brain/scene.py	23	7
ai_director/common/check.py	2	3
ai_director/common/io.py	13	0
ai_director/common/paths.py	1	2
ai_director/common/schemas.py	4	1
ai_director/director/render.py	77	18
ai_director/director/run.py	7	3
ai_director/director/scorer.py	4	1
ai_director/highlights/clips.py	80	3
ai_director/highlights/select.py	18	6
ai_director/ingest/proxy.py	36	13

File	Lines added	Lines removed
ai_director/perception/debug_video.py	5	3
ai_director/perception/faces.py	79	16
ai_director/perception/people.py	24	15
ai_director/perception/run.py	57	27
ai_director/perception/speaker.py	89	9
configs/default.yaml	52	3
eval/director_eval.py	10	1
samples/make_mock.py	4	5
samples/mock_s00/clips.json	2	1
samples/mock_s00/scores.parquet	-	-
samples/mock_s00/timeline.json	55	55
samples/mock_s00/video/cam1_640.mp4 (DELETED)	-	-
samples/mock_s00/video/cam2_640.mp4 (DELETED)	-	-
samples/mock_s00/video/cam3_640.mp4 (DELETED)	-	-
tests/test_agent.py	292	5
tests/test_asr.py	21	0
tests/test_brain.py	18	1
tests/test_contracts.py	2	1
tests/test_eval.py	1	1
tests/test_highlights.py	18	0
tests/test_perception.py	177	0
tests/test_render.py	44	0
tests/test_scorer.py	28	1
tests/test_translate.py	34	1
ai_director/agent/asks.py (NEW)	260	0
ai_director/audio/english_words.py (NEW)	195	0
ai_director/director/split.py (NEW)	174	0
tests/test_english_words.py (NEW)	122	0
tests/test_split.py (NEW)	117	0
tests/test_strip.py (NEW)	68	0

Appendix B. The full code of the new file english_words.py

Note: Arabic text inside the code is drawn right-to-left on purpose (so it can be read); the characters in the file itself are correct.

ai_director/audio/english_words.py (lines 1-70)

```
# ai_director/audio/english_words.py — Arabic sessions: the English words that Gemini Transcribe drops or writes in
# Arabic letters, put back (in English letters) by a general Gemini model that listens again.
#   python -m ai_director.audio.english_words data/sessions/s03     an existing session (then translate + captions again)
# Ahmed Salama, 2026-10-09. Gemini 3.5 Transcribe (language ar-EG) DROPS many English words ("اسمه ___ هنستخدمه!") and
# writes others in Arabic letters ("بكورسي!"); asking it for ar-EG + en-US, or auto, dropped even more. gemini-3.5-flash
# listens to each ~1-minute window again WITH that draft and writes every line, English words in English letters.
# Tested: s03 3:00-4:00 (LLM, VLM, Qwen, accuracy, GPT: confirmed by Ahmed), s01 2:00-3:00 (the Arabic stayed, only
# the English terms were added). Rows, times and speakers stay the same; only text (+ its words) changes.
import re
from concurrent.futures import ThreadPoolExecutor
from pathlib import Path

import soundfile as sf
import yaml
from pydantic import BaseModel

from ..common.io import read_json, session_from_argv, write_json
from ..common.paths import CONFIGS
from .gemini import clients, with_keys

CFG = yaml.safe_load((CONFIGS / "default.yaml").read_text(encoding="utf-8"))["audio"]
ARABIC = re.compile(r"[ء-ۿ]")
LATIN = re.compile(r"[A-Za-z]")
# A MINIMAL correction: a first prompt ("write what is said") also rewrote the Arabic (12-15 % of the words) and moved
# words to neighbouring lines (captions out of time); this one keeps them (s01 3:00-4:00: only "salary" added).
PROMPT = """This audio is part of a conversation in Egyptian Arabic. The people often use English words (technical
terms, names of models, products and companies).
Below are numbered lines: their start and end time in THIS audio, the speaker, and the draft text from a
speech-to-text system. The draft is good, BUT it often LEAVES OUT the English words, or writes them in Arabic letters.
Correct each draft line using the audio:
- Put back every English word that was said in that line, in English letters, where it was said
  (for example: accuracy, users, model, API, LLM).
- An English word written in Arabic letters (for example بكورسي, يوزرز!) -> the English word (accuracy, users).
- Keep every other word of the draft exactly as it is (same Arabic words, same spelling). Do not move words to
  another line, do not add or remove Arabic words, do not translate, do not change it to Modern Standard Arabic.
Return every numbered line with its number, also the lines that need no change.
{context}
{lines}"""


class Line(BaseModel):
    n: int
    text: str


class Lines(BaseModel):
    lines: list[Line]


def windows(rows, seconds=60.0):
    """Rows -> groups of neighbouring rows that last at most `seconds` together (a row is never cut)."""
    out, cur = [], []
    for r in rows:
        if cur and r["end"] - cur[0]["start"] > seconds:
            out.append(cur)
            cur = []
        cur.append(r)
    return out + [cur] if cur else out


def prompt_for(rows, t0, before=(), after=()):
    """The prompt for one window; times are given from the start of the window's audio (t0). before/after = the
    neighbouring lines, in the audio and the prompt as context only: a word at the edge of a window needs what was
    said just before it (s03 3:08 "Qwen", right after a window edge, was heard as "Cohere" without it)."""
    line = lambda r: f'{r["start"] - t0:.1f}-{r["end"] - t0:.1f} s, {r["speaker"]} | draft: {r["text"]}'  # noqa: E731
    context = ""
    if before or after:
        context = ("\nContext only (said just before / after these lines, do NOT return them):\n"
                   + "\n".join(f"- {line(r)}" for r in [*before, *after]) + "\n")
    return PROMPT.format(context=context, lines="\n".join(f"{i}. {line(r)}" for i, r in enumerate(rows, 1)))
```

ai_director/audio/english_words.py (lines 71-140)

```
71
72
73  def ask_flash(path, prompt, model, start=0, keyed=None):
74      """One window's audio + prompt -> ({line number: text}, key name)."""
75      def call(client):
76          f = client.files.upload(file=str(path))
77          try:
78              r = client.models.generate_content(model=model, contents=[f, prompt], config={
79                  "response_mime_type": "application/json", "response_schema": Lines, "temperature": 0,
80                  "http_options": {"timeout": 180_000}})       # ms; a hung request (seen once) -> the next key
81              return {x.n: x.text for x in r.parsed.lines} if r.parsed else {}
82          finally:
83              try:
84                  client.files.delete(name=f.name)                     # do not leave the audio on Google's side
85              except Exception:
86                  pass
87      return with_keys(call, start=start, keyed=keyed)
88
89
90  def plain(text):
91      """Arabic words written the same way: no tashkeel, one alef, ي = ى , ه = ة, no punctuation."""
92      text = re.sub(r"[-_]", "", text)
93      text = re.sub("[ٱ" ,"[آإأ", text).replace("ة" ,"ه").replace("ي" ,"ى")
94      return {w.strip(".,!?؟،") for w in text.split() if ARABIC.search(w)}
95
96
97  def plausible(draft, new):
98      """False if the new text looks like ANOTHER line's: the draft has 3+ real Arabic words (3+ letters, so not ده ,في,
99      لو) and the new text keeps less than half of them."""
100     old = {w for w in plain(draft) if len(w) >= 3}
101     return len(old) < 3 or len(old & plain(new)) >= len(old) / 2
102
103
104 def spread(row, text):
105     """The row with a new text; its words share the row's time evenly (one row = one caption, so only the row's start
106     and end matter). The old translation is removed: the translation step makes it again from the new text."""
107     ws = text.split()
108     step = (row["end"] - row["start"]) / len(ws)
109     words = [{"w": w, "start": round(row["start"] + i * step, 2), "end": round(row["start"] + (i + 1) * step, 2)}
110              for i, w in enumerate(ws)]
111     return {**row, "text": text, "translation": "", "words": words}
112
113
114 def fix_english_words(s: Path, ask=ask_flash):
115     """Arabic sessions: every transcript row's text again, with its English words (see the top). Returns how many rows
116     changed. Gemini's answers are saved in tmp/gemini_words_*.json (a re-run is free) and the rows as Gemini Transcribe
117     made them in tmp/transcript_asr.json (the draft), so running this twice gives the same result."""
118     model = CFG.get("english_words_model")
119     if not model or read_json(s / "session.json")["language"] != "ar":
120         return 0
121     rows = read_json(s / "transcript.json")
122     (s / "tmp").mkdir(exist_ok=True)
123     first = s / "tmp" / "transcript_asr.json"
124     key = lambda rs: [(r["id"], r["start"], r["end"]) for r in rs]                    # noqa: E731
125     if first.exists() and key(read_json(first)) == key(rows):
126         draft = read_json(first)                                          # already done once: the same draft again
127     else:
128         draft = rows
129         write_json(first, rows)
130     audio, sr = sf.read(s / "audio.wav", dtype="float32")
131     if audio.ndim > 1:
132         audio = audio.mean(axis=1)
133     groups = windows(draft, CFG.get("english_words_window", 60))
134     keyed = clients() if ask is ask_flash else []
135     pos = {r["id"]: i for i, r in enumerate(draft)}
136     fallback = CFG.get("english_words_fallback")      # 3.5-flash: only 20 requests/day per key (free tier)
137     main_ok = [True]                                  # once the main model fails, the other windows skip it
138
139     def one(k):
140         g = groups[k]
```

ai_director/audio/english_words.py (lines 141-195)

```
        i0, i1 = pos[g[0]["id"]], pos[g[-1]["id"]]
        before = [r for r in draft[max(i0 - 2, 0):i0] if r["end"] >= g[0]["start"] - 10]   # 2 lines of context
        after = [r for r in draft[i1 + 1:i1 + 3] if r["start"] <= g[-1]["end"] + 10]       # each side, if near
        t0, t1 = max((before or g)[0]["start"] - 0.5, 0.0), (after or g)[-1]["end"] + 0.5
        prompt = prompt_for(g, t0, before, after)
        raw = s / "tmp" / f"gemini_words_{k:03d}.json"
        if raw.exists():                                  # saved answer for the same prompt (= the same draft lines)
            saved = read_json(raw)
            if saved.get("prompt") == prompt and saved.get("model") in (model, fallback):
                return {int(n): t for n, t in saved["lines"].items()}
        path = s / "tmp" / f"words_piece_{k:03d}.wav"
        sf.write(path, audio[int(t0 * sr):int(t1 * sr)], sr, subtype="PCM_16")
        try:
            used = model if main_ok[0] or not fallback else fallback
            try:
                lines, name = ask(path, prompt, used, start=k, keyed=keyed or None)
            except Exception as e:                        # no quota left / no key works -> the fallback model
                if used == fallback or not fallback:
                    raise
                main_ok[0] = False
                print(f"    {model} can't be used ({type(e).__name__}: {str(e)[:100]}) -> {fallback}", flush=True)
                used = fallback
                lines, name = ask(path, prompt, used, start=k, keyed=keyed or None)
        finally:
            path.unlink(missing_ok=True)
        write_json(raw, {"model": used, "prompt": prompt, "key": name,
                         "lines": {str(n): t for n, t in lines.items()}})
        print(f"    window {k + 1}/{len(groups)} ({t0:.0f}-{t1:.0f} s) done with {used} ({name})", flush=True)
        return lines

    with ThreadPoolExecutor(max_workers=max(len(keyed), 1)) as pool:   # one worker per key
        answers = list(pool.map(one, range(len(groups))))
    by_id = {r["id"]: r for r in rows}
    changed = ignored = 0
    for g, lines in zip(groups, answers):
        for i, d in enumerate(g, 1):
            new = " ".join(str(lines.get(i, "")).split())
            if not new or new == by_id[d["id"]]["text"]:
                continue
            if not plausible(d["text"], new):                          # the answer slid to a neighbouring line
                ignored += 1
                continue
            by_id[d["id"]] = spread(by_id[d["id"]], new)
            changed += 1
    out = [by_id[r["id"]] for r in rows]
    if changed:
        write_json(s / "transcript.json", out)
    english = sum(bool(LATIN.search(w)) for r in out for w in r["text"].split())
    print(f"    {changed} of {len(rows)} lines rewritten, {ignored} answers ignored (looked like another line), "
          f"{english} English words in the transcript now")
    return changed


if __name__ == "__main__":
    fix_english_words(session_from_argv())
```

Appendix C. The full code of the new file asks.py

ai_director/agent/asks.py (lines 1-70)

```
# ai_director/agent/asks.py — what the user asked. The CODE reads the language, the clip length and the caption
# languages (tested 2026-10-08: qwen3:8b copies the search result's times (~30 s) and skips "end = start + 60"; its own
# arguments are used only when the message names a length or caption languages in words this file doesn't know).
# ONE small LLM question reads who to show and how much ("mostly the guest, the hosts a little bit"): that is about
# meaning, word lists can't do it (qwen3:8b: 23/24 + 9/10 new sentences; its miss: a person who is only the topic,
# "the guest's opinion", is shown). Lengths/captions stay with the code (LLM 16/20 there, the code 19-20/20).
# Owner: Ahmed Salama
import re
from typing import Literal

from pydantic import BaseModel, Field

ARABIC, LATIN = re.compile("[ف-ي]"), re.compile("[A-Za-z]")
CJK = re.compile("[□-□□-□□-□□-□]")          # Chinese / Japanese / Korean
FRANCO = re.compile(r"\b[a-z]+[2357][a-z]+|\b[2357](?!nd\b|rd\b|th\b|st\b|min|sec)[a-z]{2,}", re.I)   # e3mel, 3an
LENGTH_WORD = re.compile(r"minute|\bmins?\b|second|\bsecs?\b|\d\s*s\b|ثوان|ثواني|ثاني|دقائق|دقايق|دقيق|de2i2|da2aye2|"
                         r"sanya|sawany", re.I)
CAPTION = re.compile(r"caption|subtitle|\bsubs?\b|تايتل ?|كابشن|ترجم|targam", re.I)
NAMES = {"ar": "Egyptian Arabic", "en": "English"}

WORDS = [   # word lengths -> "N sec" / "N min" (the order matters: "دقيقتين ونص" before "دقيقتين")
    (r"\b(?:a|one)?\s*minute and a half|\bone and a half minutes?", "90 sec"),
    (r"\bhalf (?:a )?minute", "30 sec"),
    (r"\b(?:a|an|one) minute", "1 min"),
    (r"\b([نصف ?و|نص ?و:?) دقيقتين(?)(وبل)", r"\g<1>150 sec"),                # the prefix stays: "2ل" <- "لدقيقتين min"
    (r"\b([نصف ?و|نص ?و:?) [ةدقيق](?)(وبل)", r"\g<1>90 sec"),
    (r"\b([ةدقيق]نص|نص:?)(?)(وبل)", r"\g<1>30 sec"),                    # "بنص دقيقة" (أطول بنص دقيقة) too
    (r"\b([[ان|ين:?)دقيقت(?)(وبل)", r"\g<1>2 min"),
    (r"(?<!\d)(?<!\d )\b([ةدقيق](?)[وبل](?!\s*\d)", r"\g<1>1 min"),     # "كليب دقيقة" (not "5 الدقيقة" = a time)
    (r"\bde2i2a w ?nos", "90 sec"), (r"\bnos de2i2a", "30 sec"), (r"\bde2i2t[ae]+[iy]?n", "2 min"),
    (r"(?<!\d)(?<!\d )\bde2i2a", "1 min"),
]
NUMBERS = {"one": 1, "two": 2, "three": 3, "four": 4, "five": 5, "ten": 10, "fifteen": 15, "twenty": 20, "thirty": 30,
           "forty": 40, "forty-five": 45, "fifty": 50, "sixty": 60, "ninety": 90,
           "أربع": 4, "اربع": 3, "ثلاثة": 3, "تلاته": 3, "ثلاث": 3, "تلات": 3, "اثنين": 2, "اتنين": 2,
           "أربعة": 4, "خمس": 5, "خمسة": 5, "خمسه": 5, "عشر": 10, "عشرة": 10, "عشرين": 20,
           "ثلاثين": 30, "اربعين": 40, "أربعين": 40, "خمسين": 50, "ستين": 60, "تسعين": 90}
UNIT = r"(minutes?|mins?|seconds?|secs?|s|ثوان|ثواني|ثاني[ةه]|دقايق|دقائق|دقيق[ةه]|da2aye2|sanya|sawany)"
MINUTES = r"(?:minutes?|mins?|دقايق|دقائق|دقيق[ةه]|da2aye2)"
NUM = r"(?<![\d:.])(\d+(?:\.\d+)?)"
SEP = r"\s*(?:-|–|—|to|or|and|إلى|الى|او|أو|ال|ـــل|lel|le|aw)\s*"
LENGTH = re.compile(rf"{NUM}(?:\s*{UNIT})?(?:{SEP}{NUM})?\s*-?\s*{UNIT}(?!\w)")
CLOCK = re.compile(rf"(?<![\d:.])(\d{{1,2}})[:.]([0-5]\d)\s*-?\s*{MINUTES}(?!\w)")    # "1:20 / 1.20 minutes" = 80 s
PLUS = re.compile(rf"{NUM}\s*{MINUTES}\s*(?:,|and|و)?\s*{NUM}\s*(?:seconds?|secs?|s|ثوان|ثواني|ثاني[ةه]|sanya)(?!\w)")


def language(message):
    """ar" or "en": the script most WORDS are written in ("make a clip about الذكاء الاصطناعي" = English, a tie =
    Arabic). Franco-Arabic (e3mel clip 3an ...) counts as Arabic."""
    words = message.split()
    ar, en = sum(bool(ARABIC.search(w)) for w in words), sum(bool(LATIN.search(w)) for w in words)
    return "ar" if (ar and ar >= en) or FRANCO.search(message) else "en"


def reply_language(text):
    """The language of an answer, without ids, files and numbers (SPEAKER_01, clips/c1.mp4, 16:9)."""
    return language(re.sub(r"\S*[\d_/\\]\S*", " ", text))


def asked_length(message):
    """The clip length the user typed, as (shortest, longest) seconds, or None.
    "1-2 minutes" -> (60, 120), "a 30-second clip" -> (30, 30), "120 ,120) <- "اعمل كليب دقيقتين(.
    Minutes AND seconds are added: "1.20 minutes" (2 digits after the point = seconds, like a clock; "1.5 minutes"
    stays 90 s), "1:20 minutes", "1 minute 20 seconds", "80 ,80) <- "دقيقة و20 ثانية")."""
    m = message.lower().translate(str.maketrans("٠١٢٣٤٥٦٧٨٩", "0123456789"))
    m = CLOCK.sub(lambda x: f"{int(x[1]) * 60 + int(x[2])} sec", m)
    for pattern, new in WORDS:
        m = re.sub(pattern, new, m)
    m = re.sub(r"\b(" + "|".join(sorted(NUMBERS, key=len, reverse=True)) + r")\b",
               lambda x: str(NUMBERS[x.group(1)]), m)
```

ai_director/agent/asks.py (lines 71-140)

```
    m = PLUS.sub(lambda x: f"{float(x[1]) * 60 + float(x[2]):g} sec", m)
    secs = []
    for n1, u1, n2, u2 in LENGTH.findall(m):
        per = lambda u: 60 if u.startswith(("min", "دق", "da2")) else 1
        secs.append(float(n1) * per(u1 or u2))
        if n2:
            secs.append(float(n2) * per(u2))
    return (min(secs), max(secs)) if secs else None


def asked_captions(message):
    """The caption languages the user asked for: "ar", "en", "ar+en", "none", or None (not asked: the default)."""
    m, found = message.lower(), None
    for hit in CAPTION.finditer(m):
        rest = m[hit.end():]
        before = " ".join(m[:hit.start()].split()[-3:])                    # 3 words before, 4 after the word
        near = before + " " + " ".join(rest[re.match(r"\S*", rest).end():].split()[:4])
        if re.search(r"\b(?:no|without|remove|drop)\b(?:\s+\w+){0,2}\s*$|(?:بدون|من غير|مش عايز|مش عاوز|بلاش|شيل)"
                     r"\s*(?:\S+\s*)?$", before):
            return "none"
        ar = re.search(r"arabic|3|عربيarab", near)
        en = re.search(r"english|انجليز|إنجليز|انجلش|انكليز|englizy", near)
        if (ar and en) or re.search(r"\bboth\b|bilingual|اللغتين|الاثنين|الإثنين|الاتنين", near):
            return "ar+en"
        found = found or ("ar" if ar else "en" if en else None)
    return found


def default_captions(lang):
    """No caption languages asked: an English request gets Arabic + English, an Arabic one Arabic only."""
    return "ar+en" if lang == "en" else "ar"


VERTICAL = re.compile(r"vertical|portrait|tik ?tok|\breels?\b|\bshorts\b|\bstor(?:y|ies)\b|instagram|9\s*[:x/]\s*16|"
                      r"عمودي|طولي|ريل|تيك ?توك|تيكتوك|ستوري|شورتس|انستا|إنستا"|\b2amoudy", re.I)
HORIZONTAL = re.compile(r"horizontal|landscape|youtube(?!\s*shorts)|16\s*[:x/]\s*9|عرضي|يوتيوب|أفقي(?!\s*(شورتس)|"
                        r"\bofo2y", re.I)


def asked_format(message):
    """The format the user asked for, read by the code (2026-10-09: "make a short clip" became 9:16): vertical words
    (TikTok, reel, Shorts, story, 9:16" <- (عمودي, ريل", horizontal ones (YouTube, 16:9, 16:9" <- (أفقي", else None."""
    v, h = VERTICAL.search(message), HORIZONTAL.search(message)
    return "9:16" if v and not h else "16:9" if h and not v else None


CLIP_NOUN = (r"(?:clips?|reels?|shorts?|videos?|tiktoks?|highlights?|مقطع|مقاطع)?(?:ريل|كليب)?(?:ات|س)?(?:ز|ات|س)?"
             r"(فيديو|فيديوهات)?(?:تيك ?توك"")
DUAL = re.compile(r"(كليبين|ريلين|مقطعين|فيديوهين")


def asked_count(message):
    """How many clips the user asked for (2026-10-09: "make a reel about ..." made 3 clips). "3 clips", "two reels",
    "3 كليبات" ,"كليبين" -> that number; a plural without a number ("some clips", "كليبات") -> None (up to
    agent.max_clips); anything else -> 1."""
    m = message.lower().translate(str.maketrans("٠١٢٣٤٥٦٧٨٩", "0123456789"))
    if DUAL.search(m):
        return 2
    words = "|".join(sorted(map(re.escape, NUMBERS), key=len, reverse=True))
    unit = r"(?:minutes?|mins?|seconds?|secs?|دقيق|دقايق|ثاني|ثواني)\b"            # "a 2 minute clip" = 1 clip
    hit = re.search(rf"(?<![\d:.])(\d+|{words})\s+(?:(?!{unit})\w+\s+)?{CLIP_NOUN}(?!\w)", m)
    if hit:
        n = hit[1]
        return int(n) if n.isdigit() else NUMBERS[n]
    plural = r"\b(?:clips|reels|shorts|videos|tiktoks|highlights)\b|فيديوهات|مقاطع|ريلات|ريلز|كليبات"
    return None if re.search(plural, m) else 1


LONGER = re.compile(r"\blonger\b|\bextend|\blengthen|اطول|طول|أطول|زود|\batwal\b|\bzawed\b", re.I)
SHORTER = re.compile(r"\bshorter\b|\bshorten|\btrim\b|اقصر|قصر|أقصر|قلل|\ba2sar\b|\b2a[s9]ar\b", re.I
)
```

ai_director/agent/asks.py (lines 141-210)

```
DELTA = re.compile(r"\bby\b[^.]{0,20}?(?:\d|minute|second|half)|"                    # longer by 30 s / half a minute
                   r"(?:\d|minutes?|seconds?|mins?|secs?)[^\d]{0,12}?\b(?:longer|shorter|more|less)\b|"   # 30 s longer
                   r"(?:زود|زود|إقلل|إقلل|(?:أطول|اطول|أقصر|اقصر)\s*(?:بـ)?\s*(?:\S+\s+)?"            # زود 30 ثانية,
                   r"(?:[\dنص|أطول|دقيق|ثاني|ثواني|ثوان|[٠-٩]", re.I)                                  # أطول بنص دقيقة

def length_change(message):
    """A follow-up that changes a clip's length, read by the code: "make it longer" -> ("longer", None);
    "30 seconds longer", "longer by 30 s", "أطول بنص دقيقة", "زود 30 ثانية" -> ("longer", 30) (the number is how much
    to add or remove). None: no change asked ("make it 2 minutes" is a new length: asked_length)."""
    kind = "longer" if LONGER.search(message) else "shorter" if SHORTER.search(message) else None
    if not kind:
        return None
    amount = asked_length(message) if DELTA.search(message) else None
    return kind, amount[0] if amount else None


ROLES = {"en": {"host": "the host", "co-host": "the co-host", "guest": "the guest", "interviewer": "the interviewer",
                "interviewee": "the interviewee", "moderator": "the moderator", "panelist": "the panelist",
                "presenter": "the presenter", "audience": "someone in the audience"},
         "ar": {"host": "المقدم", "co-host": "المقدم المشارك", "guest": "الضيف", "interviewer": "المحاور",
                "interviewee": "الضيف", "moderator": "مدير الحوار", "panelist": "المشارك", "presenter": "مقدم العرض",
                "audience": "واحد من الجمهور"}}


def people_labels(speakers, lang):
    """{SPEAKER id: how to call that person in a reply}: the name if there is one (often blank) in the reply's letters,
    else the role ("the guest" / "الضيف"), numbered when two such people share it ("panelist 1"), else "speaker 2" /
    "المتحدث 2". An Arabic reply uses the role for a name in English letters ("Islam" became "الإسلام" = the religion)."""
    def role(p):
        r = ROLES[lang].get((p["role"] or "").lower())
        return r if r else None

    def named(p):
        script = ARABIC if lang == "ar" else LATIN
        return bool(p["name"]) and (bool(script.search(p["name"])) or not role(p))
    plain = [role(p) for p in speakers if not named(p)]
    labels, seen = {}, {}
    for k, p in enumerate(speakers, 1):
        if named(p):
            labels[p["id"]] = p["name"]
            continue
        r = role(p)
        if r and plain.count(r) == 1:
            labels[p["id"]] = r
            continue
        seen[r] = seen.get(r, 0) + 1
        base = r if r else ("speaker" if lang == "en" else "المتحدث")
        if lang == "en" and base.startswith("the "):
            base = base[4:]
        labels[p["id"]] = f"{base} {seen[r] if r else k}"
    return labels


def name_people(text, labels):
    """SPEAKER_xx in an answer -> the person's name or role; "(SPEAKER_00)" after a name is simply removed."""
    text = re.sub(r"\s*\(\s*SPEAKER_\d+\s*\)", "", text)
    return re.sub(r"SPEAKER_\d+", lambda m: labels.get(m[0], m[0]), text)


def title_problem(title, lang, said):
    """Why a clip title is not good enough ("" = fine). said = what is said in the clip (its transcript text)."""
    if CJK.search(title):
        return "the title has Chinese letters"
    ar, en = len(ARABIC.findall(title)), len(LATIN.findall(title))
    if lang == "en" and ar:
        return "the user wrote in English: write the title in English (no Arabic letters)"
    if lang == "ar" and en > ar:
        return "the user wrote in Arabic: write the title in Arabic"
    words = re.findall(r"\w+", title.lower())
```

ai_director/agent/asks.py (lines 211-260)

```
    if len(words) >= 4 and " ".join(words) in " ".join(re.findall(r"\w+", said.lower())):
        return "the title is copied from the transcript: write a catchy title that says what the clip is about"
    if len(words) > 10:
        return "the title is too long: at most 8 words"
    return ""


class FocusWish(BaseModel):                    # the LLM's answer to "who to show and how much?"
    person: str = Field(description="who the user wants on screen: a role (guest, host, ...) or a name; "
                                    "'not_asked' if nobody")
    level: Literal["not_asked", "only", "mostly", "more"]
    share: int | None = Field(None, description="a percentage the user typed for that person (70% -> 70), else null")


FOCUS_PROMPT = """You read ONE request for a video clip and say WHO the user wants to see on screen and HOW MUCH.
Only a request to SHOW / FOCUS ON / ZOOM ON a person counts ("focus on", "show", "يظهر ... خلي" ,"ركز على", "rakez 3ala").
A person who is only part of the TOPIC is NOT a request to show them ("where the guest explains ...", "the guest's opinion", "the host asks the guest", "اللي الضيف بيتكلم فيه") -> "not_asked".
- "focus on the guest", "ركز على الضيف", "show the host", "only the guest", "الضيف بس" -> "only"
- "mostly the guest", "most of the time", "أغلب الوقت", "but also the others a little bit" -> "mostly"
- "show the guest more", "show Islam more", "a bit more of the guest", "زود الضيف شوية" -> "more"
- a percentage for that person ("the guest 70% of the time") -> "mostly" and share 70
Examples:
"make a clip about salaries" -> {{"person": "not_asked", "level": "not_asked", "share": null}}
"clip where the host asks the guest about his salary" -> {{"person": "not_asked", "level": "not_asked", "share": null}}
"اعمل كليب دقيقة عن الشغل وركز على المذيع" -> {{"person": "host", "level": "only", "share": null}}
"30 second reel about AI, mostly the guest" -> {{"person": "guest", "level": "mostly", "share": null}}
"clip about CVs, show the host more" -> {{"person": "host", "level": "more", "share": null}}
"reel about jobs with the guest on screen 80% of the time" -> {{"person": "guest", "level": "mostly", "share": 80}}
Request: {message}"""


def focus_wish(answer, message):
    """The LLM's answer -> {"person", "level", "share"}; level None = nobody asked for. A share counts only if the user
    really typed that number (the LLM can't invent one)."""
    level = None if answer.level == "not_asked" or answer.person.strip().lower() in ("", "not_asked") else answer.level
    share = answer.share if answer.share and 10 <= answer.share <= 100 and re.search(
        rf"(?<!\d){answer.share}(?!\d)", message) else None
    return {"person": answer.person if level else "", "level": level, "share": share if level else None}


def read_focus(message, model, keep_alive="2m", num_ctx=8192):
    """Who the user wants on screen and how much: ONE short LLM question (~0.4 s), think=False for qwen3.
    num_ctx MUST be the agent's: with other options Ollama reloads the model at every switch (measured: 4 s each)."""
    from .search import ollama_client  # only here: the tests use a fake reader and need no Ollama
    extra = {"think": False} if model.startswith("qwen3") else {}
    r = ollama_client().chat(model=model, messages=[{"role": "user", "content": FOCUS_PROMPT.format(message=message)}],
                             format=FocusWish.model_json_schema(), keep_alive=keep_alive,
                             options={"temperature": 0, "num_ctx": num_ctx}, **extra)
    return focus_wish(FocusWish.model_validate_json(r.message.content), message)
```

Appendix D. The full code of the new file split.py

ai_director/director/split.py (lines 1-70)

```
# ai_director/director/split.py — split screen: the two close-ups side by side (16:9 only), SOMETIMES (2026-10-09)
# Rule A, fast back-and-forth: the person on one close-up and the person on the other take turns quickly.
# Rule B, the listener reacts: someone on a close-up talks while there is laughter / the AI Brain sees a reaction.
# A split shot is a normal cuts.json shot with 2 extra fields: {"start", "end", "cam": the main camera,
# "split": [left cam, right cam], "split_x": [face position of each, 0..1]} -> older readers just see "cam".
# Owner: Ahmed Salama. Tested 2026-10-09 on s01-s03 against hand labels (transcript lines + frames): turns <= 5 s +
# the mouth check = 12/16 moments right, 13/13 real fast exchanges found; Rule B 4/4 right (s01 only).
import json

import numpy as np
import pandas as pd
import yaml

from ai_director.common.io import read_json
from ai_director.common.paths import CONFIGS

CFG = yaml.safe_load((CONFIGS / "default.yaml").read_text(encoding="utf-8"))["split"]
FACES = 0.5                     # both close-ups must show a face in at least half of the moment (nobody there: no split)


def cam_turns(diar, spk_map, pair):
    """The talk as camera turns: speech on the SAME close-up camera in a row is one turn (two people on one camera are
    one shot). A speaker on no close-up (or the wide camera) is a turn with cam None (it breaks a back-and-forth)."""
    out = []
    for d in sorted(diar, key=lambda d: d["start"]):
        cam = spk_map.get(d["speaker"])
        cam = cam if cam in pair else None
        if out and cam is not None and out[-1]["cam"] == cam and d["start"] - out[-1]["end"] <= CFG["max_gap"]:
            out[-1]["end"] = max(out[-1]["end"], d["end"])
        else:
            out.append({"cam": cam, "start": d["start"], "end": d["end"]})
    return out


def fast_talk(turns):
    """Rule A: runs of at least min_turns turns that switch camera every time, each turn <= turn_max s, pauses
    <= max_gap s, the run >= min_seconds long. -> [[turn, ...], ...]"""
    runs, cur = [], []

    def close():
        if len(cur) >= CFG["min_turns"] and cur[-1]["end"] - cur[0]["start"] >= CFG["min_seconds"]:
            runs.append(list(cur))
    for t in turns:
        ok = t["cam"] is not None and t["end"] - t["start"] <= CFG["turn_max"]
        if ok and cur and t["cam"] != cur[-1]["cam"] and t["start"] - cur[-1]["end"] <= CFG["max_gap"]:
            cur.append(t)
        else:
            close()
            cur = [t] if ok else []
    close()
    return runs


def mouth_agrees(feats, run, pair):
    """The mouth check (vision, independent of who the audio says talks): in most turns of >= 0.6 s the talker's
    camera shows more mouth movement than the other close-up. Catches wrong speaker labels (s01 405 s: 1 of 3)."""
    votes = []
    for t in run:
        if t["end"] - t["start"] < 0.6:
            continue
        m = {c: feats[(feats.cam == c) & feats.t.between(t["start"], t["end"])].mouth_motion.dropna() for c in pair}
        if all(len(x) for x in m.values()):
            other = pair[1] if t["cam"] == pair[0] else pair[0]
            votes.append(m[t["cam"]].mean() > m[other].mean())
    return not votes or sum(votes) > len(votes) / 2


def reactions(events, scene, diar, spk_map, pair):
    """Rule B: the seconds with laughter (audio >= split.laugh) or a scene line that says laughter / reaction, while a
    person on one of the close-ups talks -> [(start, end)], +-react_pad s, at least min_seconds long."""
```

ai_director/director/split.py (lines 71-140)

```
    times = {e["t"] for e in events if e["laugh"] >= CFG["laugh"]}
    times |= {int(r["t"]) + k for r in scene if r.get("event") == "laughter" or r.get("raw_mode") == "reaction"
              for k in range(5)}                                          # a scene line = 5 s
    talk = [d for d in diar if spk_map.get(d["speaker"]) in pair]
    out = []
    for t in sorted(times):
        if not any(d["start"] <= t + 0.5 <= d["end"] for d in talk):
            continue
        if out and t - out[-1][1] <= 2:
            out[-1][1] = t + 1
        else:
            out.append([t, t + 1])
    pad, least = CFG["react_pad"], CFG["min_seconds"]
    res = []
    for a, b in out:
        a, b = a - pad, b + pad
        if b - a < least:                                                 # too short: grow around the middle
            a, b = (a + b - least) / 2, (a + b + least) / 2
        res.append((a, b))
    return res


def face_at(feats, diar, spk_map, cam, a, b):
    """Where to crop camera `cam` in a..b (0..1): its face while ITS person talks (perception follows the talker on a
    camera with two people: s01 cam2 = host at 0.80 or the man in grey at 0.18), else its face over a..b, else 0.5."""
    rows = feats[(feats.cam == cam) & feats.t.between(a, b) & (feats.face_area > 0)]
    own = np.zeros(len(rows), bool)
    for d in diar:
        if spk_map.get(d["speaker"]) == cam and d["end"] > a and d["start"] < b:
            own |= rows.t.between(d["start"], d["end"]).to_numpy()
    x = rows.face_x[own] if own.any() else rows.face_x
    return round(float(x.median()), 3) if len(x) else 0.5


def split_moments(s):
    """Every split moment of a session: [{"start", "end", "cam", "split", "split_x", "why"}]. [] when splits are off,
    the session has no 2 close-up cameras or a file is missing (diarization, speaker_map, features)."""
    if not CFG["enabled"]:
        return []
    meta = read_json(s / "session.json")
    close = sorted(c["id"] for c in meta["cameras"] if c["role"] == "closeup")
    need = [s / "diarization.json", s / "speaker_map.json", s / "features.parquet"]
    if len(close) != 2 or not all(f.exists() for f in need):
        return []
    pair = CFG["order"] if sorted(CFG["order"] or []) == close else close   # left, right
    diar, spk_map = read_json(s / "diarization.json"), read_json(s / "speaker_map.json")
    feats = pd.read_parquet(s / "features.parquet", columns=["t", "cam", "face_area", "face_x", "mouth_motion"])
    found = [(r[0]["start"], r[-1]["end"], "fast talk") for r in fast_talk(cam_turns(diar, spk_map, pair))
             if not CFG["mouth_check"] or mouth_agrees(feats, r, pair)]
    events = read_json(s / "audio_events.json") if (s / "audio_events.json").exists() else []
    scene = ([json.loads(x) for x in (s / "scene.jsonl").read_text(encoding="utf-8").splitlines() if x.strip()]
             if (s / "scene.jsonl").exists() else [])
    found += [(a, b, "reaction") for a, b in reactions(events, scene, diar, spk_map, pair)]
    joined = []
    for a, b, why in sorted(found):                                      # close moments become one (no flicker)
        if joined and a - joined[-1][1] < CFG["join_gap"]:
            joined[-1] = (joined[-1][0], max(joined[-1][1], b), joined[-1][2] if joined[-1][2] == why else "both")
        else:
            joined.append((a, b, why))
    out = []
    for a, b, why in joined:
        a, b = max(0.0, a), min(meta["duration"], b)
        seen = {c: (feats[(feats.cam == c) & feats.t.between(a, b)].face_area > 0).mean() for c in pair}
        if min(np.nan_to_num(list(seen.values()))) < FACES:
            continue                                                     # someone left the frame
        talk = {c: sum(max(0, min(b, d["end"]) - max(a, d["start"])) for d in diar if spk_map.get(d["speaker"]) == c)
                for c in pair}
        out.append({"start": round(a, 2), "end": round(b, 2), "cam": max(talk, key=talk.get), "split": list(pair),
                    "split_x": [face_at(feats, diar, spk_map, c, a, b) for c in pair], "why": why})
    return out
```

ai_director/director/split.py (lines 141-174)

```
141
142
143 def add_splits(cuts, moments, min_shot):
144     """The switcher's cut list with every split moment put in: the shots inside become one split shot; a piece of a
145     shot shorter than min_shot left next to a split joins the split (no flash)."""
146     out = [dict(c) for c in cuts]
147     for m in moments:
148         keep = []
149         for c in out:
150             if c["end"] <= m["start"] or c["start"] >= m["end"]:
151                 keep.append(c)
152                 continue
153             if c["start"] < m["start"]:
154                 keep.append({**c, "end": m["start"]})
155             if c["end"] > m["end"]:
156                 keep.append({**c, "start": m["end"]})
157         shot = {k: m[k] for k in ("start", "end", "cam", "split", "split_x")}
158         out = sorted(keep + [shot], key=lambda c: c["start"])
159     for i, c in enumerate(out):                                          # short leftovers join the split
160         if c.get("split"):
161             continue
162         near = [j for j in (i - 1, i + 1) if 0 <= j < len(out) and out[j].get("split")]
163         if near and c["end"] - c["start"] < min_shot:
164             j = near[0]
165             out[j]["start"], out[j]["end"] = min(out[j]["start"], c["start"]), max(out[j]["end"], c["end"])
166             c["gone"] = True
167     out = [c for c in out if not c.get("gone")]
168     merged = []
169     for c in out:                                                        # the same camera twice in a row = 1 shot
170         if merged and not c.get("split") and not merged[-1].get("split") and merged[-1]["cam"] == c["cam"]:
171             merged[-1]["end"] = c["end"]
172         else:
173             merged.append(c)
174     return [{**c, "start": round(c["start"], 2), "end": round(c["end"], 2)} for c in merged]
```